

“Contract Guardian AI: Enterprise AI-Powered Contract Risk Analysis and Polyglot Microservices Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

[P.PRANAW REDDY] :[2520030136]

[P.ANUDEEP] :[2520030087]

[P.ASRITHA CHOWDARY] :[2520030592]

Under the Esteemed Guidance of

[Dr. Y . Subbarayudu]

[Designation]

Department of Computer Science Engineering

Team ID: [2] | Section: [7]

[DBS & DBE]

[2026-27] | [Trimester]



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Contract Guardian AI: Enterprise AI-Powered Contract Risk Analysis and Polyglot Microservices Platform**” is a record of Bonafide work of **[P.PRANAW REDDY] - [2520030136], [P.ANUDEEP] - [2520030087], [P. ASRITHA] - [2520030592]** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

[P.PRANAW REDDY] – [2520030087]

[P. ANUDEEP] – [2520030087]

[P. ASRITHA] – [2520030592]

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled “**Contract Guardian AI: Enterprise AI-Powered Contract Risk Analysis and Polyglot Microservices Platform**” is a bonafide work done and submitted by **[P.PRANAW REDDY] - [2520030136], [P.ANUDEEP] - [2520030087], [P. ASRITHA] - [2520030592]** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), KLH Bachupally Campus, during the academic year **[ACADEMIC YEAR]**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of this project.

Our greatest appreciation goes to our project guide **[Dr. Y . Subbarayudu]**, Department of Computer Science Engineering, for his/her tremendous support, encouragement and guidance throughout the design, implementation, testing and documentation of Contract Guardian AI.

We express our gratitude to the Head of the Department and the faculty members of the **Department of Computer Science Engineering** for providing the facilities and academic environment required to complete this project-based laboratory.

We thank all teaching and non-teaching staff members, classmates and friends who assisted us directly or indirectly during our work.

[P.PRANAW REDDY] – [2520030087]

[P. ANUDEEP] – [2520030087]

[P. ASRITHA] – [2520030592]

Abstract

Contracts allocate money, risk and obligation between the parties who sign them, yet most signatories cannot afford a professional review of every agreement. Risky clauses such as uncapped liability, one-sided indemnities and automatic renewals, and deceptive features such as blank references and hidden obligations, are easily overlooked. This project presents Contract Guardian AI, an enterprise-grade, AI-powered contract analysis and risk mitigation platform.

Users upload legal agreements as PDF, DOCX or TXT files. The platform extracts the text, performs semantic chunking with overlap, detects high-risk clauses using GPT-4o-mini with a heuristic fallback, identifies fraudulent patterns, computes a risk score from 0 to 100 across 26 risk flag categories, suggests safer clause alternatives, answers questions through a retrieval-augmented generation pipeline and generates PDF audit reports. Real-time analytics are provided through advanced SQL queries.

The system is built as a polyglot microservices architecture: a React and TypeScript frontend, a .NET Core 8 API gateway, a Python FastAPI core backend, a Node.js activity service and a Spring Boot report service. PostgreSQL with pgvector serves as the relational database and vector store, and MongoDB stores activity logs and cached analyses. Security uses JWT access tokens with refresh token rotation and role-based access control. The platform is containerised with Docker Compose, described for Kubernetes, delivered through a GitHub Actions pipeline and instrumented with Prometheus and OpenTelemetry, demonstrating all six course outcomes (CO1 to CO6) in one system.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-11
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	12-16
3	Technology Stack	3.1 Front-end (React, TypeScript, Vite) 3.2 Back-end (FastAPI, Node.js, Spring Boot, .NET Core) 3.3 Database (PostgreSQL, pgvector, MongoDB) 3.4 Version Control (Git, GitHub) 3.5 APIs or External services (OpenAI)	17-21
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (cloud, local, CI/CD pipelines)	22-25
5	Design	5.2 ER Diagram / Database schema	26-29
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	30-36
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	37-41
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	42-47
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	48-52
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	53-57

S. No.	Chapters	Topics	Page.no
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	58-60
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	61-64
13	References	- Books, tutorials, APIs, documentation sites used	65-66
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	67-73

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Contracts govern almost every commercial relationship. An employment offer, a software licence, a vendor agreement, a lease and a loan are all contracts, and each one allocates money, risk and obligation between the parties who sign it. The difficulty is that the party who drafts a contract and the party who signs it rarely have the same understanding of what it says. Contracts are long, they are written in specialised language, and the clauses that matter most are often the ones that are easiest to overlook: a liability clause without a cap, an indemnity that runs in one direction only, a renewal that happens automatically unless notice is given months in advance, or a reference to a schedule that was never attached.

Large organisations manage this risk with legal departments. Individuals, freelancers, start-ups and small businesses usually cannot. They sign agreements they have not fully read, or have read but not fully understood, because a professional review of every document is too slow and too expensive. The consequence is discovered later, when a fee appears that nobody expected, a termination turns out to be impossible, or a dispute must be settled in a jurisdiction on the other side of the world.

Recent progress in large language models has made it possible to read and reason about legal text automatically. A model can be asked to identify the clauses of a contract, to say which of them are unusually one-sided and to propose fairer wording. At the same time, advances in vector databases have made it practical to search a document by meaning instead of by keyword, so that a question such as "what happens if I cancel early" can be answered from the relevant passages even when those passages never use the word "cancel".

Contract Guardian AI is an enterprise-grade, AI-powered contract analysis and risk mitigation platform built on these capabilities. A user uploads a legal agreement as a PDF, DOCX or TXT file, and the system extracts and parses the text, divides it into semantically meaningful chunks, detects high-risk clauses, identifies deceptive or fraudulent patterns, computes a risk score from 0 to 100, suggests safer alternatives for the risky clauses and generates a PDF audit report. Real-time analytics over all of a user's contracts are provided through advanced SQL queries.

The platform is also an engineering exercise. It is built as a polyglot microservices architecture: four independent services written in four different languages and frameworks, backed by a relational database, a document database and a vector store, packaged in containers and instrumented for monitoring. This structure was chosen so that a single project demonstrates all six outcomes of the course, from relational database design through NoSQL and vector databases, backend APIs, microservices and system design patterns to DevOps and observability.

The text of a contract is extracted with PyMuPDF for PDF files and python-docx for DOCX files. The primary analysis path uses the GPT-4o-mini model through LangChain. Because an external model is not always available, the platform also contains a heuristic analysis engine based on regular-expression patterns, which produces a result of the same shape without any external service. The risk assessment covers 26 risk flag categories, and a separate set of checks looks for signs of deception such as blank references, hidden obligations and impossible timelines.

The background of the project can be summarised through the areas of computer science on which it draws:

1. Relational Database Engineering: a normalised PostgreSQL schema with constraints, indexes, views, triggers, stored functions and procedures, and advanced SQL including window functions and common table expressions.
2. NoSQL and Vector Databases: MongoDB for flexible activity and cache documents, and pgvector for embedding storage and approximate nearest-neighbour search.
3. Backend Engineering: an asynchronous FastAPI application with validated contracts, token-based authentication, role-based access control and structured error handling.
4. Distributed Systems: independent services in Python, Node.js, Java and C# communicating over REST and WebSocket behind an API gateway.
5. System Design: caching, rate limiting, the circuit breaker pattern and asynchronous background processing.
6. DevOps and Observability: Docker Compose, Kubernetes manifests, a GitHub Actions pipeline, Prometheus metrics and OpenTelemetry tracing.
7. Applied Artificial Intelligence: prompt-driven analysis with a large language model, embeddings and retrieval-augmented generation.

The remainder of this chapter states the problem the platform addresses, the boundaries of what was built, the objectives that guided the work, the importance of the application and the users for whom it is intended.

1.2 Problem Statement

The central problem is that people sign contracts whose risks they cannot see. The risk in a contract is not announced; it is distributed through the document in clauses that look routine. Reading for risk requires knowing what to look for, and most signatories do not.

The first part of the problem is volume and time. A reviewer must read every clause of every agreement. For an organisation that handles many vendor, customer and employment contracts, manual review becomes a bottleneck, and under time pressure documents are skimmed or not reviewed at all.

The second part is expertise. Recognising that an indemnification clause is unusually broad, that a non-compete restriction is excessive or that a payment term is one-sided requires legal experience. Without it, a reader may understand every sentence and still miss the significance of what they have read.

The third part is deception. Some contracts are not merely unfavourable but misleading. They refer to annexures that are blank or missing, leave key terms undefined, omit signatures, impose obligations in unexpected places, set timelines that cannot be met or attach penalties that are out of proportion. These patterns are difficult to notice precisely because they consist of what is absent or inconsistent.

The fourth part is the lack of a measurable result. Even when a review is carried out, its conclusion is usually an opinion. There is no consistent score that allows one contract to be compared with another, no record of which clauses were judged risky and why, and no way to see how the risk across a portfolio of contracts is changing over time.

The fifth part is the limitation of keyword search. A user who wants to know what a contract says about a topic can search for a word, but legal drafting uses many different words for the same idea. A search that depends on exact wording misses relevant passages.

There is also an engineering side to the problem. A system that analyses contracts must store structured results, unstructured logs and high-dimensional vectors, each of which suits a different kind of database. It must call an external AI model that is slow, costly and occasionally unavailable, without becoming slow, costly or unavailable itself. It must protect confidential documents with proper authentication. And it must be observable, so that its behaviour in operation can be measured.

Contract Guardian AI addresses these problems together. It reads the contract automatically, applies a consistent set of risk categories, detects deceptive patterns, expresses the result as a score and a risk level, proposes safer wording, records everything in a queryable form and allows the document to be searched by meaning. When the language model cannot be reached, it falls back to a deterministic heuristic engine so that the user still receives an analysis.

1.3 Scope of the Project

The scope of the project covers the complete path from the upload of a document to the delivery of an audit report, together with the supporting services, databases and infrastructure.

Document ingestion is within scope. The platform accepts PDF, DOCX and TXT files, computes a SHA-256 hash of each upload to detect duplicates, extracts the text and stores the contract with its metadata. Contract versions are recorded, and contracts can be tagged.

Analysis is within scope. The platform splits a contract into clauses, analyses them with GPT-4o-mini or with the heuristic engine, assigns risk flags from 26 categories, detects fraud indicators, computes a risk score between 0 and 100 and derives a risk level of LOW, MEDIUM, HIGH or CRITICAL. It can rewrite an individual clause into a safer alternative.

Semantic search and retrieval-augmented generation are within scope. Contract text is chunked, embedded and stored in pgvector. Users can run similarity searches, hybrid keyword and vector searches, and questions answered by the language model from the retrieved passages.

Reporting and analytics are within scope. The platform generates PDF audit reports and exposes analytics endpoints built on advanced SQL: joins, subqueries, common table expressions, window functions, aggregates and full-text search.

Security is within scope: account registration and login, JWT access tokens, refresh token rotation, password hashing, three roles and rate limiting.

The microservices are within scope. The system comprises a Python FastAPI core backend, a Node.js activity service with real-time streaming over Socket.io, a Spring Boot report service and a .NET Core 8 API gateway with JWT validation, routing and a circuit breaker. A React single-page application provides the user interface.

Infrastructure is within scope: Dockerfiles for each service, a Docker Compose stack, Kubernetes manifests, a GitHub Actions CI/CD pipeline, Prometheus metrics, Grafana dashboards and optional OpenTelemetry tracing.

Some matters are outside the scope and should be stated clearly. The platform does not provide legal advice; its output is an aid to review and does not replace a qualified lawyer. Scanned documents that contain only images are not processed by optical character recognition. Kubernetes support is provided as manifests only and a running cluster deployment is not claimed. Adapters for the Pinecone and Weaviate vector stores exist as stubs and are not full integrations. A payments table exists in the schema, but integration with a payment provider is not part of the implemented system.

1.4 Objective(s)

The primary objective of the project is to design and build a platform that analyses a legal contract automatically and tells the user, clearly and consistently, where its risks lie.

A second objective, of equal weight in the academic context, is to demonstrate all six course outcomes in one coherent system instead of six separate exercises. Each outcome is realised by a part of the platform that has a genuine purpose in it.

The specific objectives are the following:

1. To design a PostgreSQL schema in third normal form with primary keys, foreign keys, CHECK and UNIQUE constraints and enumerated types, and to demonstrate ACID transactions, views, triggers, stored functions and procedures, and B-Tree, GIN and HNSW indexes (CO1).
2. To use MongoDB for activity logs, analysis caching and conversation sessions, and pgvector for embeddings and cosine similarity search, and to implement text chunking and a retrieval-augmented generation pipeline (CO2).
3. To build a RESTful backend in FastAPI with Pydantic v2 validation, JWT access tokens, refresh token rotation, structured errors with request identifiers, OpenAPI documentation and an automated test suite (CO3).
4. To implement a polyglot microservices architecture with services in Python, Node.js, Java and C#, each with its own responsibilities, dependencies and Dockerfile (CO4).

5. To apply system design patterns: caching of analysis results, token bucket rate limiting, a circuit breaker in the gateway and asynchronous background tasks (CO5).

6. To package and operate the system with Docker Compose, Kubernetes manifests, a CI/CD pipeline, Prometheus metrics and OpenTelemetry tracing (CO6).

7. To detect high-risk clauses and deceptive patterns in contracts, score the risk, suggest safer wording and produce a PDF audit report.

8. To keep the platform usable when the external language model is unavailable, through a heuristic fallback.

The mapping between the course outcomes and their implementation is summarised in Table 1.1.

CO	Title	Implementation in the platform
CO1	Relational DBMS and Advanced SQL	PostgreSQL schema in 3NF; constraints; ACID transactions; window functions, CTEs, subqueries, joins; PL/pgSQL triggers, views, functions and procedures; B-Tree, GIN and HNSW indexes
CO2	NoSQL and Vector Databases	MongoDB through the Motor async driver; pgvector with HNSW cosine search; chunking; OpenAI and TF-IDF embeddings; RAG pipeline; Pinecone and Weaviate adapter stubs
CO3	Backend Architecture and RESTful APIs	FastAPI; Pydantic v2; JWT access and rotating refresh tokens; pbkdf2_sha256 hashing; structured errors; OpenAPI; pytest suite
CO4	Microservices Architecture	FastAPI, Node.js/Express with Socket.io and Mongoose, Spring Boot, and .NET Core 8 with EF Core
CO5	System Design and Enterprise Patterns	Analysis caching; token bucket rate limiting; circuit breaker; background tasks; asyncio concurrency
CO6	DevOps, Cloud and Observability	Docker Compose; Kubernetes manifests; GitHub Actions; Prometheus; OpenTelemetry

Table 1.1: Course outcome mapping

1.5 Importance of the Application

The application is important first for the people it serves. A freelancer who is sent a client agreement, a small business owner presented with a supplier contract or an employee reading an offer letter can upload the document and learn within a short time which clauses deserve attention. The platform does not make the decision for them, but it tells them where to look and what a fairer version of the clause might say.

It is important for consistency. A human reviewer's attention varies; the platform applies the same 26 risk categories and the same thresholds to every document. Two contracts can be compared by their scores, and a portfolio of contracts can be ranked.

It is important for transparency. Every analysis is stored with its clauses, its findings, its score and its level, and every significant action is recorded in an audit trail. A user or an auditor can see what was analysed, when, by whom and with what result.

It is important for the detection of deception. By looking specifically for blank references, missing signatures, undefined terms, hidden obligations, impossible timelines and excessive penalties, the platform draws attention to features that a hurried reader would pass over.

It is important as a demonstration of resilience in AI-backed systems. Applications that depend on a language model inherit its cost, latency and availability. Contract Guardian AI shows three ways of managing that dependency: caching results by document hash so that the same document is not analysed twice, providing a heuristic fallback so that analysis continues without the model, and protecting the gateway with a circuit breaker so that one failing service does not bring down the rest.

It is important academically. The project shows why different kinds of data belong in different kinds of database: structured, related data with integrity requirements in PostgreSQL; flexible, high-volume log documents in MongoDB; and embedding vectors in pgvector. It shows the relational database doing much more than storing rows, through triggers that write the audit trail, views that rank risk and functions that perform full-text search. And it shows the operational side of software, namely containers, pipelines, metrics and tracing, which is often left out of academic projects.

1.6 Target Users / Audience

The platform defines three roles: USER, ANALYST and ADMIN. Each corresponds to a kind of person who uses the system.

The USER role is for the individuals and business users who upload their own contracts. They register, sign in, upload agreements, run analyses, read the results, ask questions about their documents, download reports and review their history. They see only their own contracts.

The ANALYST role is for people whose work is the review of contracts, such as legal analysts, compliance officers and procurement staff. They use the same analysis features with additional access granted through the gateway's analyst-or-admin policy.

The ADMIN role is for those who operate the platform. Administrators have access to administrative functions protected by the admin-only policy and to the system-wide analytics.

Beyond these roles, the platform has several wider audiences. Small and medium enterprises without an in-house legal team can use it as a first line of review. Legal professionals can use it to triage documents so that their time is spent on the clauses that matter. Students and teachers of database systems and distributed systems can study it as a worked example in which each course outcome appears in a realistic setting. Developers can study it as a reference for combining a relational database, a document database and a vector store behind a set of cooperating services.

The frontend is a web application, so the only requirement on the user's side is a modern browser. Private pages are available only after sign-in, and the server enforces the user's role on every request.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements describe the physical resources needed to develop, run and demonstrate the platform. Contract Guardian AI is heavier than a typical web application because it is not one program but a group of them. In its full configuration it runs a React frontend, a Python backend, a Node.js service, a Java service, a .NET gateway, PostgreSQL, MongoDB, Prometheus and Grafana. The hardware must be sized for all of these running together.

For development, a modern multi-core 64-bit processor is required. The services run concurrently, and several of them, in particular the Java virtual machine and the .NET runtime, have significant start-up and memory costs. A processor with four or more cores keeps the system responsive while containers are built and tests are run.

Memory is the most important resource. Each container holds its own runtime, and the two databases keep working data in memory. 8 GB of RAM is a workable minimum when only the core backend, the frontend and the databases are run. 16 GB is recommended for running the complete Docker Compose stack together with an IDE and a browser.

Solid-state storage is recommended. Container images for five different technology stacks, the dependency directories of the Python, Node.js, Java and .NET projects and the database volumes together occupy several gigabytes, and building images involves a great deal of small-file input and output. An SSD shortens build and start-up times considerably.

A network connection is required to pull base images and install dependencies, and at run time to reach the OpenAI API when the language-model analysis path and OpenAI embeddings are used. When no API key is configured, the platform uses its heuristic analysis and TF-IDF embeddings and can run without external network access.

No graphics processor is needed. The language model and the embedding model are accessed as remote services, and the local fallback methods are light computations on the processor.

For a deployed environment, the servers must provide enough memory and processing capacity for the containers, and durable storage for the PostgreSQL and MongoDB volumes and for uploaded files. The project does not state a production sizing, because the system was not load tested at production scale. On the client side, any device with a current web browser is sufficient. The requirements are summarised in Table 2.1.

Component	Minimum	Recommended
Processor	64-bit dual-core	Four cores or more
Memory	8 GB RAM (core services only)	16 GB RAM (full Compose stack)
Storage	SSD with 20 GB free	SSD with 40 GB or more free
Network	Connection for images and packages	Stable broadband; access to the OpenAI API
Graphics	Not required	Not required
Client device	Any device with a modern browser	Desktop or laptop display

Table 2.1: Hardware requirements

2.2 Software Requirements

The software requirements are grouped by the part of the system to which they belong. Because the platform is polyglot, it needs the runtimes of four languages, unless it is run entirely in containers, in which case Docker supplies them.

The core backend requires Python 3.11 or later with FastAPI, SQLAlchemy 2.0 in asynchronous mode, Alembic, Pydantic v2, python-jose and passlib for authentication, LangChain and the OpenAI client for the language model, PyMuPDF and python-docx for document parsing, ReportLab for PDF generation, Motor for MongoDB and the pgvector client. In production it is served by Gunicorn with Uvicorn workers.

The activity service requires Node.js 20 with Express 4, Mongoose, Socket.io, Winston, express-rate-limit, helmet and morgan. The report service requires Java 17 with Spring Boot 3 and Maven. The API gateway requires the .NET 8 SDK with ASP.NET Core, Entity Framework Core with the Npgsql provider and the JwtBearer authentication package.

The frontend requires Node.js to build and run. It uses React 18.3.1 with TypeScript 5.7, Vite 6.0.5, React Router DOM 7, Axios 1.7, React Query 3, Framer Motion 11, Lucide React and Tailwind CSS 3.4.

Two database servers are required. PostgreSQL 16 is used with the pgvector, pg_trgm and pgcrypto extensions; the project uses the pgvector/pgvector:pg16 image, in which the vector extension is already installed. MongoDB 7 is used for document storage.

Docker and Docker Compose are required to run the complete stack. Prometheus and Grafana run as containers for monitoring. Git is required for version control. An OpenAI API key is optional: with it the platform uses GPT-4o-mini and the text-embedding-3-small model, and without it the platform uses its fallbacks. The software and versions are listed in Table 2.2.

Layer	Technology	Version
Frontend framework	React	18.3.1
Frontend language	TypeScript	5.7.2
Frontend build	Vite	6.0.5

Layer	Technology	Version
CSS framework	Tailwind CSS	3.4.19
Server state	React Query	3.39.3
HTTP client	Axios	1.7.9
Animation	Framer Motion	11.15.0
Primary backend	Python FastAPI	0.115+ on Python 3.11+
ORM	SQLAlchemy (async)	2.0+
Migrations	Alembic	1.13+
Validation	Pydantic	v2.7+
Authentication	python-jose, passlib	3.3+, 1.7.4+
AI / LLM	LangChain, OpenAI	0.2+, 1.35+
PDF parsing	PyMuPDF	1.24+
PDF generation	ReportLab	4.2+
Async MongoDB driver	Motor	3.4+
Vector extension client	pgvector	0.3+
Activity service	Node.js, Express	20, 4
Report service	Spring Boot (Java 17)	3
API gateway	.NET Core / ASP.NET Core	8
Relational database	PostgreSQL (pgvector image)	16
Document database	MongoDB	7
Tracing	OpenTelemetry	1.25+
Containerisation	Docker, Docker Compose	Compose file format 3.9

Table 2.2: Software requirements and versions

2.3 Development Tools and Frameworks

FastAPI is the framework of the core backend. It was chosen for its asynchronous request handling, which suits a service that spends much of its time waiting for a database or for an external model, and for its integration with Pydantic, which validates every request and response. FastAPI also generates OpenAPI documentation with an interactive Swagger interface from the declared routes and models.

SQLAlchemy 2.0 provides the object-relational mapping, used here through its asynchronous engine and AsyncSession. Alembic manages schema migrations for the application. In addition, the database directory contains plain SQL scripts for the schema, the indexes, the views, triggers and functions, and the advanced query demonstrations; these are mounted into the PostgreSQL container and executed when the database is first initialised.

Pydantic v2 defines the request and response models. The models use camelCase aliases, so that JSON exchanged with the frontend follows JavaScript naming conventions while the Python code follows Python conventions.

Pydantic Settings loads the configuration, which comprises more than thirty parameters, from environment variables.

LangChain is used to structure the interaction with the language model through prompt templates, and the OpenAI client provides access to GPT-4o-mini and to the embedding model. Motor is the asynchronous driver through which the backend uses MongoDB without blocking the event loop.

Express is the framework of the activity service, with Mongoose as the object-document mapper for MongoDB and Socket.io for real-time delivery of events to connected browsers. Winston provides structured logging, helmet sets protective HTTP headers and express-rate-limit restricts request rates.

Spring Boot with Spring MVC is the framework of the report service, built with Maven. ASP.NET Core is the framework of the gateway, with Entity Framework Core for database access and the JwtBearer middleware for token validation.

On the frontend, Vite provides the development server and the production build, TypeScript provides static typing with strict checking, Tailwind CSS provides utility-first styling with a custom set of design tokens, and React Query manages data fetched from the server, including caching and refetching.

Testing uses pytest with pytest-asyncio for the backend and Jest for the Node.js service. A custom Python verification script checks the six course outcomes with live assertions. The TypeScript compiler and the Vite build serve as the verification of the frontend.

Docker builds an image for each service from its own Dockerfile, and Docker Compose defines how the containers are networked, in what order they start and how their health is checked. Kubernetes manifests describe the same system for a cluster. GitHub Actions runs the pipeline of linting, testing, image building, pushing and deployment. Prometheus collects metrics from the backend and Grafana displays them.

Git and GitHub are used for version control and collaboration. The services, the SQL scripts, the infrastructure definitions and the frontend are kept in one repository, so that a single commit can change a service together with its tests and its deployment configuration.

The functional requirements of the system are that it must register and authenticate users, accept and parse contract files, detect duplicates, analyse contracts and their clauses, score and classify risk, detect fraud indicators, rewrite clauses, embed and search contract text, answer questions through retrieval-augmented generation, record activity, produce analytics and generate PDF reports.

The non-functional requirements are data integrity, enforced by the constraints of the relational schema; security, through hashed passwords, short-lived tokens and role checks; resilience, through the fallback engine, the cache and the circuit breaker; responsiveness, through asynchronous input and output and background tasks; observability, through metrics, request identifiers and tracing; and portability, through containers.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End (React, TypeScript, Vite)

The frontend of Contract Guardian AI is a single-page application built with React 18.3.1 and TypeScript 5.7 and bundled with Vite 6.0.5. It is the part of the platform that the user sees: the pages for signing in, uploading contracts, reading analyses, downloading reports and reviewing history.

React provides the component model. Each page is composed of smaller components, such as cards, tables, badges, upload zones and charts, which are written once and reused. A component is a function of the data it is given, so that when an analysis result arrives from the server the page renders it without any manual manipulation of the document.

TypeScript adds static types to the application. The shape of every API response is declared as a type, and the compiler checks that the components use those shapes correctly. Mistakes such as reading a field that does not exist or passing a string where a number is expected are found at build time. The project is compiled in strict mode, and a clean type check is part of its verification.

Vite serves the application during development with fast hot-module replacement and produces the optimised static build for production. In the Docker Compose stack, the built files are served by Nginx on port 8080; during development the Vite server runs on port 5173.

React Router DOM 7 handles client-side routing. The application has ten pages. Three are public: the landing page, the login page and the registration page. Seven are private: the dashboard, upload, analysis, report, history, profile and settings pages. The private pages are wrapped in a PrivateRoute component that checks whether the user is authenticated and redirects to the login page if not. The routes are listed in Table 3.1.

Route	Page	Access
/	LandingPage	Public
/login	LoginPage	Public
/register	RegisterPage	Public
/dashboard	DashboardPage	Private (JWT required)
/upload	UploadPage	Private
/analysis	AnalysisPage	Private
/report	ReportPage	Private
/history	HistoryPage	Private
/profile	ProfilePage	Private
/settings	SettingsPage	Private

Table 3.1: Application routes

Axios 1.7 is the HTTP client. It sends requests to the API with the user's JSON Web Token attached. React Query 3 manages server state on top of it: it caches the results of requests, tracks whether a request is loading or has failed, and refetches data when it becomes stale. Components therefore do not manage loading flags and caches by hand.

The token is stored in the browser's localStorage, and an isAuthenticated check guards the private routes. This check is a matter of user experience. The protection of the data lies on the server, which validates the token and the role on every request.

Styling uses Tailwind CSS 3.4 with PostCSS and Autoprefixer. The design is a dark theme defined through custom design tokens in the Tailwind configuration. The background colours are ink (#070b14) and panel (#0d1424), with a translucent frost colour for glass-like surfaces. A legal accent palette supplies gold (#d9b76f), teal (#35d3c8), blue (#7aa7ff), red (#ff5f6d), amber (#ffb454) and green (#48d597). These colours carry meaning in the interface: red and amber mark higher risk, and green marks low risk. Custom shadows give a teal glow to highlighted elements and a red one to dangerous findings, and a grid pattern forms the background. The typeface is Inter.

Framer Motion 11 provides animation for page transitions and for the appearance of results, and Lucide React provides the icons. Together these give the interface a polished character appropriate to a product intended for professional users.

3.2 Back-End (FastAPI, Node.js, Spring Boot, .NET Core)

The server side of the platform consists of four services in four technologies. Each was chosen for the task it performs.

The core backend is written in Python with FastAPI and listens on port 8000. It is responsible for authentication, contract management, analysis, analytics, vector search and the retrieval pipeline. Python is the natural choice for this service because the libraries for document parsing, language models and embeddings are Python libraries. FastAPI's asynchronous design allows the service to wait for the database, MongoDB and the OpenAI API concurrently instead of holding a thread idle for each request.

The backend is organised in layers. The api package contains the routers, one for each area: auth, contracts, analysis, analytics, vector, activity, reports and health. The services package contains the business logic: the AI analysis service, the clause detection service, the vector store service, the authentication service, the contract service, the document parser, the activity service, the analysis orchestration service and the report service. The repositories package contains the database access, following the repository pattern with classes such as UserRepository and ContractRepository. The schemas package contains the Pydantic models, the db package contains the SQLAlchemy models, the session factory and the MongoDB client, and the core package contains configuration, constants, security functions, error types and logging.

The application entry point is an application factory that assembles the middleware, the routers, the start-up and shutdown handlers, the Prometheus endpoint and the error handlers. The middleware stack consists of CORS handling with configurable origins, a request-metrics middleware that adds X-Response-Time-MS and X-Request-ID headers to every response, Prometheus instrumentation and, optionally, OpenTelemetry instrumentation.

All errors are returned in one JSON envelope containing a code, a message, optional details and the request identifier. Three global handlers produce it: one for the application's own typed errors, one for request validation failures and one for unexpected exceptions.

The activity service is written in Node.js 20 with Express 4 and listens on port 3001. It ingests activity events, stores them in MongoDB through Mongoose and streams them to connected browsers through Socket.io. Node.js suits this service because its event-driven model handles many simultaneous lightweight connections well and Socket.io is a mature library for real-time communication.

The report service is written in Java 17 with Spring Boot 3 and listens on port 8082. It is responsible for enterprise PDF report generation, the storage of report metadata and a REST API for retrieving reports. Spring Boot represents the kind of service commonly found in enterprise environments and shows that the architecture can incorporate one.

The API gateway is written in C# with .NET 8 and ASP.NET Core and listens on port 5000. It is the single entry point for clients. It validates the JSON Web Token centrally, applies role-based policies, routes requests to the appropriate upstream service, propagates request identifiers and protects the upstream services with a circuit breaker. It uses Entity Framework Core with the Npgsql provider for its database health check.

3.3 Database (PostgreSQL, pgvector, MongoDB)

The platform follows the principle of polyglot persistence: each kind of data is kept in the kind of database that suits it.

PostgreSQL 16 is the primary relational database. It holds the data that has a fixed structure, relationships and integrity requirements: users, contracts, contract versions, analyses, clause analyses, risk findings, refresh tokens, payments, activity logs, embeddings and tags. The schema is normalised to third normal form. It uses UUID primary keys generated by the pgcrypto extension, foreign keys with defined delete behaviour, CHECK constraints on values such as risk scores, UNIQUE constraints on values such as email addresses and storage paths, and enumerated types for roles, statuses, risk levels, file types and activity types. JSONB columns hold the semi-structured parts of an analysis.

PostgreSQL is used well beyond table storage. Views present dashboard metrics, risk rankings and audit trails. PL/pgSQL triggers maintain timestamps and write audit records automatically. Stored functions and a stored procedure perform statistics, search and cleanup. B-Tree indexes support ordinary lookups, and GIN indexes support full-text and trigram search over contract text and titles.

pgvector is a PostgreSQL extension that adds a vector data type and similarity operators. The `contract_embeddings` table stores a 1536-dimensional vector for each chunk of each contract. An HNSW index, built with the parameters `m=16` and `ef_construction=64` and using cosine distance, makes nearest-neighbour search fast. Keeping the vectors in PostgreSQL means that they sit beside the contracts they describe, are deleted with them through the foreign key and can be combined with ordinary SQL conditions in a single query.

MongoDB 7 is the document database. It is accessed from the Python backend through the asynchronous Motor driver and from the Node.js service through Mongoose. It holds three collections in the backend: `activity_logs` for flexible activity documents, `analysis_cache` for cached analysis results keyed by the SHA-256 hash of the contract, and `rag_sessions` for the history of question-and-answer conversations. These documents vary in shape, are written at high frequency and are read mostly by simple keys, which is the pattern for which a document database is suited.

MongoDB documents refer to PostgreSQL records by identifier; for example, an activity document carries the `contract_id` of a contract stored in PostgreSQL. The two databases are thus linked at the application level.

Uploaded files themselves are kept in file storage, with the storage path recorded in the contracts table.

3.4 Version Control (Git, GitHub)

Git is used for version control and GitHub hosts the repository. The repository is organised by component: a backend directory for the FastAPI application, a frontend directory for the React application, a services directory containing the Node.js activity service and the .NET gateway among others, a database directory containing the SQL scripts, a kubernetes directory containing the manifests, and the Docker Compose file at the root.

Keeping all services in one repository suits a project of this size. A change that affects the contract between two services, for example a new field returned by the backend and displayed by the frontend, can be made in one commit and reviewed as a whole.

GitHub is also the home of the continuous integration and delivery pipeline. The workflow file in the `.github/workflows` directory defines a pipeline that runs on changes to the repository: it lints the code, runs the pytest suite, builds the Docker images, pushes them to a registry and deploys.

Secrets are not committed. The JWT secret, the database credentials and the OpenAI API key are supplied through environment variables, and in the Kubernetes manifests through Secret objects.

3.5 APIs or External Services

The platform exposes its own REST API and uses one external service.

The API of the core backend is versioned under `/api/v1` and is divided into groups: authentication, contracts, analysis, analytics, vector search and retrieval, activity, reports and health. It is documented automatically in OpenAPI format with a Swagger interface. Chapter 6 lists the endpoints in detail.

The external service is the OpenAI API. Two of its models are used. GPT-4o-mini performs the contract analysis, the rewriting of clauses and the answering of questions in the retrieval pipeline. The text-embedding-3-small model converts text into 1536-dimensional vectors for semantic search. The models are configured through environment variables, so they can be changed without altering the code.

LangChain sits between the application and the model. It provides the ChatPromptTemplate used to build prompts and a consistent interface to the model. The analysis request asks for a JSON object as the response format, and the response is validated against a Pydantic model before it is used.

The dependence on an external service is managed carefully. If no API key is configured, the analysis is performed by the heuristic engine and embeddings are produced by a TF-IDF method. Results are cached by document hash so that the model is not called twice for the same content.

Socket.io provides the real-time channel between the Node.js activity service and the frontend. It is a library within the platform, not an external service, but it is the second communication protocol in the system alongside REST.

For vector storage the platform uses pgvector. Adapter stubs for Pinecone and Weaviate are included to show how an alternative managed vector store could be substituted; they are not complete integrations.

Prometheus and Grafana are used for monitoring and run as containers within the stack. The complete stack is summarised in Table 3.2.

Layer	Technology	Role
Presentation	React, TypeScript, Vite, Tailwind CSS	Single-page user interface
Gateway	.NET 8, ASP.NET Core	JWT validation, routing, circuit breaker
Core backend	Python, FastAPI, SQLAlchemy, Pydantic	Auth, contracts, analysis, analytics, vector search
Activity service	Node.js, Express, Mongoose, Socket.io	Activity logs and real-time events
Report service	Java, Spring Boot	PDF report generation
Relational data	PostgreSQL 16	System of record
Vector data	pgvector (HNSW)	Embeddings and similarity search
Document data	MongoDB 7	Activity logs, analysis cache, RAG sessions
AI	GPT-4o-mini, text-embedding-3-small, LangChain	Analysis, rewriting, embeddings, RAG
Operations	Docker, Kubernetes, GitHub Actions	Packaging, orchestration, CI/CD
Observability	Prometheus, Grafana, OpenTelemetry	Metrics, dashboards, tracing

Table 3.2: Summary of the technology stack

4.1 High-level architecture diagram

```

graph TD
    ReactSPA["React SPA  
(TypeScript, Vite) :5173 / :8080"]
    Gateway[".NET Core 8 API Gateway :5000  
JWT validation · routing · circuit breaker"]
    FastAPI["FastAPI Core Backend :8000  
auth · contracts · analysis  
analytics · vector / RAG"]
    Nodejs["Node.js / Express :3001  
activity logs  
Socket.io real-time"]
    SpringBoot["Spring Boot :8082  
enterprise PDF  
report generation"]
    PostgreSQL["PostgreSQL 16  
+ pgvector :5432"]
    FileStorage["File Storage"]
    MongoDB["MongoDB 7 :27017"]
    OpenAI["OpenAI API  
GPT-4o-mini · embeddings  
(heuristic fallback)"]
    Observability["Observability and delivery:  
Prometheus :9090 · Grafana :3000 · OpenTelemetry · Docker Compose · Kubernetes manifests · GitHub Actions CI/CD"]

    ReactSPA -- "HTTP (REST, Bearer JWT)" --> Gateway
    ReactSPA -.->|WebSocket (Socket.io)| Gateway
    Gateway --> FastAPI
    Gateway --> Nodejs
    Gateway --> SpringBoot
    FastAPI --> PostgreSQL
    FastAPI --> FileStorage
    FastAPI -.-> OpenAI
    Nodejs --> MongoDB
    Observability
  
```

The diagram illustrates the architecture of a React SPA application. At the top, the **React SPA** (TypeScript, Vite) runs on ports 5173 and 8080. It communicates with the **.NET Core 8 API Gateway** (port 5000) via HTTP (REST, Bearer JWT). A dashed arrow indicates a **WebSocket (Socket.io)** connection between the React SPA and the API Gateway. The API Gateway routes requests to three backend services: **FastAPI Core Backend** (port 8000), **Node.js / Express** (port 3001), and **Spring Boot** (port 8082). The FastAPI Core Backend interacts with **PostgreSQL 16 + pgvector** (port 5432), **File Storage**, and the **OpenAI API** (GPT-4o-mini, embeddings, heuristic fallback). The Node.js / Express service interacts with **MongoDB 7** (port 27017). At the bottom, a bar represents the **Observability and delivery** layer, including Prometheus (9090), Grafana (3000), OpenTelemetry, Docker Compose, Kubernetes manifests, and GitHub Actions CI/CD.

The platform follows a polyglot microservices style: four independent services in four different languages and frameworks. The React single-page application sends HTTP requests to the .NET Core 8 API gateway. The gateway validates the token and routes each request to the FastAPI core backend, the Node.js activity service or the Spring Boot report service. The core backend uses PostgreSQL with pgvector, MongoDB and file storage. The ports and purposes of the components are listed in Table 4.1.

Service	Technology	Port	Purpose
Frontend	React 18, TypeScript, Vite	5173 (dev) / 8080 (prod)	Single-page user interface
Core backend	Python FastAPI	8000	Analysis, auth, contracts, analytics, vector
Activity service	Node.js / Express	3001	Activity logs, real-time Socket.io, MongoDB
Report service	Spring Boot (Java)	8082	Enterprise PDF report generation
API gateway	.NET Core 8 / ASP.NET	5000	Request routing, JWT validation
PostgreSQL + pgvector	pg16	5432	Relational database and vector store
MongoDB	Mongo 7	27017	Activity logs and analysis cache
Prometheus	Prometheus	9090	Metrics scraping
Grafana	Grafana	3000	Metrics dashboards

Table 4.1: Services and ports

4.2 Description of Each Layer (Frontend, Backend, Database)

The architecture can be described as a set of layers through which a request passes: the frontend, the gateway, the services and the data stores.

The frontend layer is the React application running in the user's browser. It renders the pages, collects input, sends requests and displays results. It holds the user's token and attaches it to each request. It also maintains a Socket.io connection to the activity service, through which it receives events as they happen, so that the interface can show live activity without polling.

The gateway layer is the .NET Core 8 service. It is the single address to which clients send requests. For each request the gateway validates the JSON Web Token with the JwtBearer middleware, applies the authorisation policy for the route, which is either open to any authenticated user, restricted to analysts and administrators or restricted to administrators only, and forwards the request to the correct upstream service through a named HttpClient. It attaches an X-Request-ID header, so that the request can be followed through the services, and an X-Gateway header identifying itself.

The gateway also aggregates health information, exposing a health endpoint that includes a database check through Entity Framework Core. And it implements the circuit breaker. In the CLOSED state requests pass through normally. When failures of an upstream service exceed a threshold, the circuit moves to OPEN and requests to that service fail immediately instead of waiting for timeouts. After an interval the circuit moves to HALF-OPEN and allows a probe request; if it succeeds the circuit returns to CLOSED. This prevents the failure of one service from exhausting the resources of the gateway and cascading to the others.

The service layer contains the three upstream services. The FastAPI core backend holds most of the business logic. Within it, a request passes through the middleware, reaches a router, is validated by a Pydantic schema, and is handled by a service class that uses repositories to reach the database. The Node.js activity service receives activity events through its REST API, stores them and broadcasts them. The Spring Boot report service generates enterprise PDF reports and stores report metadata.

Each service owns its data layer. The FastAPI backend uses PostgreSQL and MongoDB. The Node.js service uses MongoDB. The gateway reaches PostgreSQL through Entity Framework Core for its health check. No service reaches into the internals of another; they communicate through their APIs.

The data layer consists of PostgreSQL with pgvector, MongoDB and file storage. PostgreSQL is the system of record and enforces the integrity of the structured data. pgvector, inside PostgreSQL, holds the embeddings. MongoDB holds the activity logs, the analysis cache and the retrieval sessions. File storage holds the uploaded documents.

Communication between the layers takes two forms. Synchronous communication uses REST over HTTP with a Bearer token; this is how the frontend talks to the gateway and how the gateway talks to the services. Real-time communication uses WebSocket through Socket.io, from the activity service to the frontend.

The flow of a contract analysis illustrates how the layers cooperate. The user selects a file on the upload page. The frontend sends it through the gateway to the contracts endpoint of the core backend. The backend computes the SHA-256 hash of the file and checks whether the same content has already been uploaded. It extracts the text with the parser appropriate to the file type, stores the file, and inserts the contract row; a database trigger records the upload in the activity log. When the user requests analysis, the backend checks the analysis cache in MongoDB by the hash. On a miss, the analysis service detects the clauses and calls the AI analysis service, which uses GPT-4o-mini or the heuristic engine. The result is stored as an analysis with its clause analyses and risk findings in PostgreSQL, where a second trigger records the completion, and is cached in MongoDB. The activity is also logged to MongoDB in a background task, so that the response is not delayed. The frontend then displays the score, the level, the flagged clauses and the suggested alternatives.

A cross-cutting observability layer accompanies these. The backend exposes a metrics endpoint that Prometheus scrapes, Grafana displays the metrics, and OpenTelemetry can trace a request through FastAPI and SQLAlchemy when enabled.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The deployment architecture is container-based. Each service has its own Dockerfile, which builds an image containing the service, its runtime and its dependencies. An image runs the same way on a developer's laptop, in the pipeline and on a server.

For local deployment, Docker Compose defines the stack. The containers join a bridge network named contract-guardian-net, on which they reach one another by container name. The stack includes cg-postgres, built from the pgvector/pgvector:pg16 image with a named volume for its data and with the SQL initialisation scripts mounted into the directory from which PostgreSQL runs scripts on first start; cg-mongodb, built from the mongo:7-jammy image with its own volume; cg-backend, which runs FastAPI under Gunicorn with two Uvicorn workers and has a health check; cg-node-activity, the Node.js activity service; cg-frontend, which serves the React build through Nginx on port 8080; cg-prometheus, with a custom scrape configuration; and cg-grafana, with a provisioning volume.

The start-up order is controlled by health checks. PostgreSQL starts first, and the next container waits until it reports healthy; then MongoDB, then the backend, the activity service, the frontend, Prometheus and Grafana. This ordering prevents a service from starting before the database on which it depends is ready to accept connections.

The production command of the backend container first applies the Alembic migrations and then starts Gunicorn with the Uvicorn worker class, bound to port 8000, with two workers and a timeout of 180 seconds. The generous timeout allows for long-running analysis requests.

For cloud deployment, Kubernetes manifests are provided in the `kubernetes/manifests` directory. They include Deployments with liveness and readiness probes, Services of the ClusterIP and LoadBalancer types, ConfigMaps for environment configuration, Secrets for sensitive credentials and an Ingress for routing external traffic. The liveness probe allows the cluster to restart a container that has stopped responding, and the readiness probe keeps traffic away from a container that is not yet ready. Kubernetes support is provided as manifests; the report does not claim a running production cluster.

The CI/CD pipeline is defined in GitHub Actions. Its stages are lint, test, build, push and deploy. The lint stage checks the code. The test stage runs the pytest suite. The build stage builds the Docker images. The push stage publishes them to a container registry. The deploy stage releases them. A failure at any stage stops the pipeline, so that code which fails its tests is not built into an image and released.

Configuration is supplied through environment variables in every environment: in a local environment file for development, in the Compose file for the local stack, and in ConfigMaps and Secrets in Kubernetes. Chapter 9 describes the deployment procedure and the configuration variables in detail.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database Schema

Contract Guardian AI — PostgreSQL ER Structure

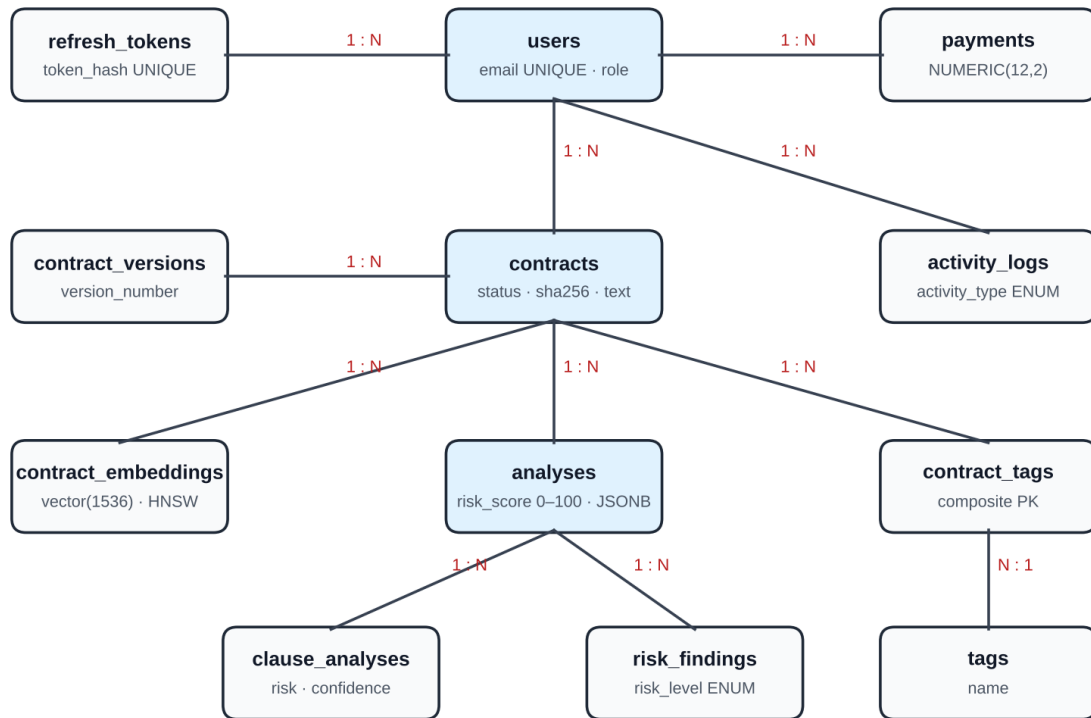


Figure 5.1: Entity-relationship structure of the PostgreSQL schema

The relational schema is normalised to third normal form. A user owns contracts; a contract has versions, analyses, embeddings and tags; an analysis has clause analyses and risk findings. The tables and their key constraints are listed in Table 5.1. The extensions used are vector (pgvector), pg_trgm for trigram search and pgcrypto for gen_random_uuid.

Table	Description	Key constraints
users	User accounts	UUID PK; UNIQUE email; CHECK (email LIKE '%@%'); role ENUM
contracts	Uploaded contracts	UUID PK; FK to users (CASCADE); UNIQUE storage_path; CHECK (file_size_bytes > 0); date consistency CHECK
contract_versions	Version history	UUID PK; FK to contracts; UNIQUE (contract_id, version_number)
analyses	AI analysis results	UUID PK; FK to contracts and users; CHECK (risk_score BETWEEN 0 AND 100); JSONB columns
clause_analyses	Individual clause results	UUID PK; FK to analyses; CHECK constraints on risk, confidence and score
risk_findings	Normalised risk findings	FK to analyses; risk_level ENUM
refresh_tokens	JWT refresh tokens	UUID PK; UNIQUE token_hash; CHECK (expires_at > created_at)
payments	Payment records	UUID PK; UNIQUE payment_id; NUMERIC(12,2); CHECK (amount >= 0)
activity_logs	Audit trail	BIGSERIAL PK; FK to users (SET NULL); activity_type ENUM
contract_embeddings	Vector embeddings	UUID PK; FK to contracts; vector(1536); UNIQUE (contract_id, chunk_index)
tags / contract_tags	Tagging (many-to-many)	Composite PK on the join table

Table 5.1: PostgreSQL tables

Five enumerated types restrict columns to fixed sets of values, as shown in Table 5.2.

ENUM type	Values
user_role	USER, ADMIN, ANALYST
contract_status	UPLOADED, PARSING, PARSED, ANALYZING, ANALYZED, ERROR
risk_level	LOW, MEDIUM, HIGH, CRITICAL
file_type	PDF, DOCX, TXT
activity_type	USER_LOGIN, USER_LOGOUT, USER_SIGNUP, CONTRACT_UPLOAD, CONTRACT_DELETE, CONTRACT_VIEW, ANALYSIS_START, ANALYSIS_COMPLETE, REPORT_GENERATE, VECTOR_SEARCH, RAG_QUERY

Table 5.2: Enumerated types

The indexing strategy uses three index types. B-Tree indexes cover users(email), contracts(user_id, status, created_at DESC), analyses(risk_level) and refresh_tokens(token_hash, expires_at). GIN indexes cover the full-text vector of contracts.extracted_text and the trigram operator class on contracts.title. An HNSW index covers contract_embeddings.embedding with cosine operators (m=16, ef_construction=64). Composite indexes cover activity_logs(entity_type, entity_id) and clause_analyses(analysis_id, position).

Views, triggers and stored routines complete the design. They are listed in Tables 5.3, 5.4 and 5.5.

View	Purpose	SQL features
v_contract_dashboard	Per-user dashboard metrics	Multi-table LEFT JOIN, GROUP BY, COUNT/AVG/MAX, FILTER clause
v_risk_summary	Risk rankings per user	RANK(), DENSE_RANK(), ROW_NUMBER() with PARTITION BY
v_clause_risk_heatmap	Clause risk bands	CASE expression for computed bands
v_activity_audit	Full audit trail	LEFT JOIN users, ORDER BY
v_analytics_overview	Monthly statistics and risk distribution	Nested CTEs, CROSS JOIN, NULLIF, window function for percentages
v_database_catalog	DBMS catalogue metadata	information_schema query, correlated subquery

Table 5.3: Views

Trigger	Event	Action
trg_users_updated_at	BEFORE UPDATE on users	Sets updated_at to NOW()
trg_contracts_updated_at	BEFORE UPDATE on contracts	Sets updated_at to NOW()
trg_log_contract_upload	AFTER INSERT on contracts	Inserts a CONTRACT_UPLOAD row into activity_logs
trg_log_analysis_complete	AFTER INSERT on analyses	Inserts an activity row with risk metadata

Table 5.4: PL/pgSQL triggers

Name	Type	Purpose
fn_user_risk_stats(UUID)	Function returning TABLE	Totals, average risk score and counts by risk level for a user
fn_top_risky_clauses(limit)	Function returning TABLE	System-wide top risky clause titles with average scores
proc_cleanup_expired_tokens()	Procedure	Transactional cleanup of expired refresh tokens with ROLLBACK on error
fn_search_contracts(UUID, TEXT, INT)	Function	Full-text search using ts_rank and plainto_tsquery
fn_isolation_demo()	Function	Documents PostgreSQL MVCC isolation levels
fn_set_updated_at()	Trigger function	Shared updated_at setter

Table 5.5: Stored functions and procedures

MongoDB complements the relational schema with three collections, listed in Table 5.6. Activity data is embedded in a single document, while contract_id refers to the contracts table in PostgreSQL.

Collection	Purpose	Indexes
activity_logs	Flexible activity log documents	(user_id, created_at DESC); (activity_type); (contract_id)
analysis_cache	AI analysis results cached by SHA-256	UNIQUE (contract_sha256); (created_at DESC); (contract_id)
rag_sessions	RAG conversation history per user and contract	UNIQUE (session_id); (user_id, contract_id); (created_at DESC)

Table 5.6: MongoDB collections

CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The implementation of Contract Guardian AI is organised into modules, each with a defined responsibility. Within the core backend the modules are the service classes; around it are the three other services. This section describes each module, what it does and how it does it.

The Document Ingestion Module handles the upload of a contract. It is implemented by the contract service and the document parser. When a file arrives, the service computes its SHA-256 hash. The hash identifies the content exactly, so that if the same document is uploaded again the duplicate is detected without comparing the files. The parser then routes the file by its type: PDF files are read with PyMuPDF, DOCX files with python-docx and TXT files by decoding their bytes as UTF-8. The extracted text is stored with the contract, the file is written to storage and its path is recorded. The contract's status moves through the values of the `contract_status` type, from `UPLOADED` through `PARSING` to `PARSED`, or to `ERROR` if the text cannot be extracted.

The Clause Detection Module divides the extracted text into clauses. It is implemented by the clause detection service, which uses regular expressions to recognise the boundaries of clauses, such as numbered headings and section titles. Analysing a contract clause by clause, and not as one block of text, allows the platform to say which particular clause is risky and to offer a replacement for that clause alone.

The AI Analysis Module is the centre of the platform. It is implemented by the AI analysis service and has two paths.

The primary path uses LangChain with the GPT-4o-mini model. A `ChatPromptTemplate` presents the contract text to the model with instructions and a JSON schema describing the expected output. The request sets the response format to a JSON object, which makes the model return structured data instead of free prose. The response is parsed with Pydantic's `model_validate`, so that a response that does not match the expected structure is rejected and never stored.

The fallback path is used when no OpenAI key is configured. It is a heuristic engine built from regular-expression patterns, each associated with a risk flag and a severity weight. If a pattern matches the text, its flag is raised. The risk score is calculated as a base of 15 plus the sum of the severity weights of the matched flags, capped at 100. The weights of the principal flags are given in Table 6.1.

Risk flag	Weight	Risk flag	Weight
UNLIMITED_LIABILITY	36	HIDDEN_FEES	16
BROAD_INDEMNIFICATION	30	ARBITRATION	16
CONSUMER_RIGHTS_VIOLATION	28	CONFIDENTIALITY_IMBALANCE	16
NON_COMPETE	22	VENDOR_LOCK_IN	16
TERMINATION_RESTRICTION	22	RISKY_PERCENTAGE	16
ONE_SIDED_PAYMENT_TERMS	20	AUTOMATIC_RENEWAL	16
IP_OWNERSHIP_ISSUE	18	JURISDICTION_RISK	14
DATA_PRIVACY_CONCERN	18	BLANK_REFERENCES	14

Table 6.1: Severity weights of the principal risk flags

As an example, a contract in which the heuristic engine finds unlimited liability, automatic renewal and a jurisdiction risk receives a score of $15 + 36 + 16 + 14 = 81$. The score is converted to a risk level by fixed thresholds: 85 and above is CRITICAL, 65 and above is HIGH, 35 and above is MEDIUM and anything lower is LOW. The example contract is therefore HIGH. The thresholds are implemented in a single function, `risk_level_from_score`, in the constants module, so that every part of the system classifies a score in the same way.

The complete set of risk flags comprises 26 categories. Among them is a group concerned with fraud and deception: `BLANK_REFERENCES`, for references to schedules or annexures that are empty; `MISSING_SIGNATURES`; `IMPOSSIBLE_TIMELINES`; `EXCESSIVE_PENALTIES`; `UNDEFINED_TERMS`; and `HIDDEN_OBLIGATION`. These identify contracts that are not only unfavourable but potentially misleading.

Both paths return a result of the same structure, so the rest of the system does not need to know which one produced it. The module also provides clause rewriting: given a risky clause, it returns a safer alternative wording.

The Analysis Orchestration Module, implemented by the analysis service, coordinates an analysis from start to finish. It sets the contract's status to `ANALYZING`, consults the cache, invokes clause detection and AI analysis, and persists the result. The result is stored in normalised form: one row in `analyses` with the overall score, level and JSONB details; one row in `clause_analyses` for each clause, with its risk, confidence and score; and rows in `risk_findings` for the individual findings. The contract's status then becomes `ANALYZED`.

The Vector Store Module, implemented by the vector store service, provides semantic search. It splits the contract text at sentence boundaries into chunks of 1000 characters with an overlap of 200 characters. The overlap ensures that a sentence or idea that falls across the boundary of two chunks is fully present in at least one of them. Each chunk is embedded with the OpenAI text-embedding-3-small model, producing a vector of 1536 dimensions. A SHA-256 hash of each chunk is used for deterministic deduplication, so that identical text is not embedded twice. When no OpenAI key is available, a TF-IDF method produces the vectors. The vectors are written to the `contract_embeddings` table in a batch upsert.

Search embeds the query in the same way and finds the nearest chunks by cosine similarity, using `pgvector`'s distance operator and the HNSW index. The number of results is configurable and defaults to five. A hybrid search

combines a keyword match using ILIKE with the vector similarity score, which helps when a query contains an exact term, such as a party name or a section number, that should be matched literally.

The retrieval-augmented generation pipeline builds on search. The question is embedded, the top chunks are retrieved, they are placed into a LangChain prompt together with the question, and GPT-4o-mini produces a structured answer grounded in those passages. The conversation is saved in the rag_sessions collection in MongoDB.

The Authentication Module, implemented by the authentication service and the security module, handles registration, login and token refresh. It is described in Section 6.4.

The Activity Module, implemented by the activity service of the backend, records what users do. It uses a dual-write strategy: the primary record is written to the activity_logs table in PostgreSQL, and a copy is written to MongoDB in a background task. Some activity rows are created by database triggers without any application code: inserting a contract or an analysis causes PostgreSQL itself to add the corresponding audit row.

The Analytics Module exposes more than fifteen endpoints that run advanced SQL against the relational data. They serve both as features of the product and as demonstrations of the first course outcome. They include a dashboard built on multi-table joins, a risk distribution using GROUP BY with a CASE ordering, rankings using ROW_NUMBER, RANK and DENSE_RANK, quartiles using NTILE(4), the top risky contracts using a common table expression, user activity using a correlated subquery, contracts with above-average risk using a subquery, users with more than two contracts using GROUP BY and HAVING, a combined overview using UNION, statistics using COUNT DISTINCT, AVG, SUM, MAX and PERCENTILE_CONT, and full-text search using to_tsvector, plainto_tsquery and ts_rank. A transaction demonstration performs a two-step insert that is rolled back on error.

The Reporting Module generates PDF audit reports. Within the backend, the report service uses ReportLab to produce a document containing the contract's details, its score and level, the flagged clauses and the suggested alternatives. The Spring Boot service provides enterprise report generation as a separate microservice.

The Node.js Activity Service is a separate module in its own process. It accepts activity events at POST /api/activity, returns a user's activity history with pagination at GET /api/activity/:userId, manages audit sessions and streams events over Socket.io. Its Mongoose schemas define an activity document with a UUID identifier, user, contract, activity type, session, description, IP address, metadata and creation time, and an audit session document with a unique session identifier, the user, start and end times, an embedded array of events and a summary. Requests pass through a chain of middleware: helmet, CORS, JSON body parsing, morgan logging through a Winston stream, a rate limit of 200 requests per minute and request-identifier injection.

The .NET API Gateway is the final module. Its implementation of token validation, routing, policies and the circuit breaker was described in Chapter 4.

6.2 Front-End Logic (UI Rendering, State Management)

The frontend is a React application written in TypeScript. Its entry component sets up the router with the ten routes and wraps the private ones in the `PrivateRoute` component.

Rendering is driven by state. A page declares what it needs from the server through React Query. While the request is in progress the page shows a loading state; on success it renders the data; on failure it shows the error returned by the server. React Query keeps the result in its cache under a key, so that returning to a page shows the data immediately while a refetch runs in the background.

Three kinds of state are distinguished. Server state, meaning data that lives on the server such as the list of contracts or an analysis, is managed by React Query. Authentication state is the token held in `localStorage`, read by the `isAuthenticated` function and attached to requests by the HTTP client. Local interface state, such as the file currently selected for upload, the active tab or the contents of a form, is held in the component with React's state hooks.

The upload page lets the user choose a PDF, DOCX or TXT file and submits it as multipart form data. It reflects the progress of the contract through its statuses. The analysis page presents the result: the overall score and level, displayed with the colour corresponding to the level; the list of clauses with their individual risk; the flags raised; and for each risky clause the suggested safer wording. The dashboard page presents aggregate figures from the analytics endpoints, such as the number of contracts and analyses and the distribution of risk. The history page lists past contracts and analyses, the report page allows a PDF to be generated and downloaded, and the profile and settings pages show and adjust the user's details.

The risk level is shown consistently across pages through the design tokens: green for LOW, amber for MEDIUM and red for HIGH and CRITICAL, with the red glow shadow reserved for dangerous findings. A reader can thus judge a contract's position at a glance before reading the details.

Framer Motion animates the transitions between pages and the appearance of result cards. The animations are short and serve to draw attention to new information.

Because the application is written in TypeScript with strict checking, the types of API responses are declared and verified. The Pydantic models of the backend use camelCase aliases, so the JSON fields received by the frontend match the naming of the TypeScript types without conversion.

6.3 API Integration

The frontend communicates with the server side through the REST API using Axios and receives live events through Socket.io. The API of the core backend is grouped by area. The principal endpoints are listed in Table 6.2.

Group	Endpoint	Purpose
Auth (/api/v1/auth)	POST /signup	Register a user; returns access and refresh tokens
Auth	POST /login	Authenticate; returns access and refresh tokens
Auth	POST /refresh	Rotate the refresh token; returns a new token pair
Auth	GET /me	Current user profile
Contracts (/api/v1/contracts)	POST /upload	Upload PDF, DOCX or TXT; SHA-256 deduplication; parse
Contracts	GET /	List the user's contracts with pagination
Contracts	GET /{id}	Contract details
Contracts	DELETE /{id}	Delete a contract (cascades to analyses and embeddings)
Analysis (/api/v1/analysis)	POST /analyze	Run AI analysis on a contract
Analysis	GET /	List analyses for a contract
Analysis	GET /{id}	Detailed analysis with clauses
Analysis	POST /rewrite-clause	Safer alternative for a clause
Analytics (/api/v1/analytics)	GET /dashboard	Multi-table JOIN statistics
Analytics	GET /risk-distribution	GROUP BY risk level with CASE ordering
Analytics	GET /window-functions	ROW_NUMBER, RANK, DENSE_RANK
Analytics	GET /cte-analysis	CTE-based top risky contracts
Analytics	GET /user-activity	Correlated subquery for user activity
Analytics	GET /trending-risks	Time-series risk trend
Analytics	GET /clause-statistics	Clause-level aggregates
Vector (/api/v1/vector)	POST /embed	Chunk and embed a contract
Vector	POST /search	Semantic similarity search
Vector	POST /rag-query	Full RAG pipeline query
Vector	GET /chunks/{contract_id}	Stored chunks of a contract
Activity (/api/v1/activity)	GET /	Activity logs from MongoDB
Activity	POST /	Log a new activity
Activity	GET /audit-sessions	RAG and audit sessions
Reports (/api/v1/reports)	POST /generate	Generate a PDF report
Reports	GET /{id}/download	Download a generated PDF
Health	GET /health	Service health: database connectivity and MongoDB ping

Table 6.2: Principal API endpoints of the core backend

Every request and response body is defined by a Pydantic v2 model. Invalid input is rejected with status 422 before it reaches a service. The interactive documentation generated from these models allows any endpoint to be tried from the browser.

Errors follow a unified JSON envelope with four fields: a machine-readable code such as EMAIL_ALREADY_REGISTERED, a human-readable message, a details object and the request identifier. The frontend uses the code to choose what to display and can show the request identifier to the user, who can quote it when reporting a problem; the same identifier appears in the server's logs.

The integration with the OpenAI API takes place only on the server. The API key is held in the backend's environment and is never sent to the browser. The backend calls the model through LangChain, validates the reply and returns its own response to the frontend.

Integration between services uses REST with the Bearer token. The gateway forwards the user's request to the upstream service along with the request identifier. The backend reaches MongoDB directly through Motor for its own collections.

Rate limiting protects the API. The FastAPI backend applies a token bucket limit of 100 requests per 60-second window, configurable through environment variables, and the Node.js service applies its limit of 200 requests per minute to routes under `/api/`. A client that exceeds the limit is refused until the window allows more requests.

6.4 Authentication & Authorization

Authentication is based on JSON Web Tokens with refresh token rotation.

When a user registers, the password is hashed with `pbkdf2_sha256` through the `passlib` library, and only the hash is stored. The scheme is configurable and can be changed to `bcrypt`. The users table enforces a unique email address and a basic format check through a `CHECK` constraint.

When a user signs in, the submitted password is verified against the stored hash. On success the system issues two tokens. The access token is a JWT signed with the `HS256` algorithm. Its payload contains the subject, which is the user's identifier, the role, a type field with the value "access" and an expiry time. It is valid for 30 minutes. The refresh token is an opaque, URL-safe random value of 48 bytes, valid for 14 days. It is not a JWT and carries no information. The database stores only its SHA-256 hash, in the `refresh_tokens` table, so that a copy of the database does not reveal usable tokens.

The access token is sent with each request. Because it is short-lived, a stolen access token is of limited use. When it expires, the client presents the refresh token to the refresh endpoint. The server hashes it, finds the matching row, checks that it has not expired or been revoked, revokes it and issues a new access token and a new refresh token. This is refresh token rotation: each refresh token can be used once. If an old refresh token is presented again, it is refused. A stored procedure, `proc_cleanup_expired_tokens`, removes expired tokens from the table within a transaction that is rolled back on error.

Authorisation is role-based. The three roles are `USER`, `ANALYST` and `ADMIN`. In the FastAPI backend, a `require_role` dependency is attached to routes that need a particular role; the dependency reads the authenticated user and refuses the request if the role is insufficient. In the gateway, the same roles are enforced through two named policies, `AdminOnly` and `AnalystOrAdmin`. The check therefore takes place both at the edge and in the service.

Ownership is enforced through the data model. A contract belongs to the user who uploaded it through the foreign key to users, and the contract endpoints return only the caller's own contracts. Deleting a user's contract cascades to its analyses and embeddings.

On the frontend, the token is kept in localStorage and private routes are guarded by the PrivateRoute wrapper. This prevents an unauthenticated visitor from seeing private pages, while the real enforcement remains on the server.

Several further measures support security. Rate limiting restricts the number of requests a client can make. The Node.js service uses helmet to set protective headers. CORS is configured with explicit origins. Secrets are supplied through the environment. And every login, logout and sign-up is written to the activity log, giving a record of authentication events. The token properties are summarised in Table 6.3.

Property	Access token	Refresh token
Form	JWT signed with HS256	Opaque 48-byte URL-safe value
Lifetime	30 minutes	14 days
Contents	sub (user id), role, type, exp	None
Stored in database	No	SHA-256 hash only
Use	Sent with every request	Exchanged once for a new pair, then revoked

Table 6.3: Access and refresh tokens

CHAPTER 7- FEATURES

7.1 List of Main Features

Contract Guardian AI provides a set of features that together take a contract from upload to audit report. The main features are the following.

The first feature is Contract Upload and Parsing. Users upload agreements as PDF, DOCX or TXT files. The platform extracts the text, detects duplicate uploads and keeps the contract with its metadata, versions and tags.

The second feature is AI Risk Analysis. The platform detects high-risk clauses using GPT-4o-mini, with a heuristic fallback, and assesses the contract against 26 risk flag categories.

The third feature is Fraud and Deception Detection. The platform identifies patterns such as blank references, missing signatures, hidden obligations, undefined terms, excessive penalties and impossible timelines.

The fourth feature is Risk Scoring. Each contract receives a score from 0 to 100 and a level of LOW, MEDIUM, HIGH or CRITICAL.

The fifth feature is Safer Clause Suggestions. For a risky clause the platform proposes alternative wording that is fairer to the user.

The sixth feature is Semantic Search and Question Answering. Users can search a contract by meaning and ask questions that are answered from the contract's own text.

The seventh feature is PDF Audit Reports. The platform generates a comprehensive report of the analysis that can be downloaded and shared.

The eighth feature is the Analytics Dashboard. Real-time analytics computed with advanced SQL show the user's contracts, analyses, risk distribution, rankings and trends.

The ninth feature is Activity Tracking and Audit. Significant actions are logged, and live activity is streamed to the interface.

The tenth feature is Secure Access. Accounts are protected by hashed passwords, short-lived access tokens, rotating refresh tokens, role-based access and rate limiting.

Supporting these are platform features that the user does not see directly but benefits from: caching of analysis results, the circuit breaker, health checks, metrics and tracing. The features are summarised in Table 7.1.

Feature	What the user gets	Implemented by
Upload and parsing	Text extracted from PDF, DOCX, TXT; duplicates detected	Contract service, document parser
AI risk analysis	High-risk clauses identified across 26 categories	AI analysis service (GPT-4o-mini / heuristics)
Fraud detection	Deceptive patterns highlighted	AI analysis service
Risk scoring	Score 0-100 and risk level	Scoring rules and thresholds
Clause rewriting	Safer alternative wording	Rewrite-clause endpoint
Semantic search and RAG	Answers grounded in the contract	Vector store service, pgvector
PDF reports	Downloadable audit report	ReportLab; Spring Boot service
Analytics	Dashboard, rankings, trends	Analytics endpoints, SQL views
Activity and audit	History and live events	Triggers, MongoDB, Node.js, Socket.io
Secure access	Protected account and data	JWT, refresh rotation, RBAC, rate limiting

Table 7.1: Main features

7.2 How Each Feature Works (User Perspective + Technical Description)

Contract Upload and Parsing

User perspective: After signing in, the user opens the upload page, chooses a contract file and submits it. The contract appears in their list with its title and status. If they upload a file they have uploaded before, the platform recognises it.

Technical description: The frontend posts the file to the upload endpoint. The contract service computes the SHA-256 hash of the content and checks it against existing contracts for deduplication. The document parser selects PyMuPDF, python-docx or UTF-8 decoding according to the file type. The file is saved to storage, and a row is inserted into contracts with a unique storage path, a positive file size enforced by a CHECK constraint and the extracted text. The AFTER INSERT trigger `trg_log_contract_upload` adds a `CONTRACT_UPLOAD` row to `activity_logs`. A GIN index on the text vector makes the contract searchable by full-text search immediately.

AI Risk Analysis and Fraud Detection

User perspective: The user selects a contract and requests analysis. After a short wait the analysis page shows the overall result and a list of the contract's clauses. Risky clauses are marked with the flags that apply to them, such as unlimited liability or automatic renewal, with an explanation. Indicators of possible deception, such as a reference to a blank schedule, are shown as well.

Technical description: The `analyze` endpoint calls the analysis service. The service first looks in the MongoDB `analysis_cache` collection for an entry with the contract's SHA-256 hash; a hit is returned without further work. Otherwise the clause detection service splits the text into clauses, and the AI analysis service analyses it. With an OpenAI key, a LangChain ChatPromptTemplate sends the text to GPT-4o-mini with JSON output enforced, and the reply is validated with Pydantic. Without a key, the regular-expression patterns are applied and the matched flags collected. The fraud flags are evaluated as part of the same analysis. The result is saved to the analyses,

clause_analyses and risk_findings tables and written to the cache. The trg_log_analysis_complete trigger records the completion with the risk metadata.

Risk Scoring

User perspective: Every analysed contract shows a number from 0 to 100 and a level in a colour: green for low, amber for medium and red for high and critical. The user can compare contracts by score and see at once which need the most attention.

Technical description: In the heuristic path the score is 15 plus the sum of the severity weights of the matched flags, capped at 100. The level is derived by risk_level_from_score: CRITICAL at 85 or above, HIGH at 65 or above, MEDIUM at 35 or above and LOW otherwise. The analyses table enforces the range of the score with a CHECK constraint and the level with the risk_level enumerated type, so that an out-of-range score or an invalid level cannot be stored by any path.

Safer Clause Suggestions

User perspective: Beside a risky clause the user sees a suggested alternative. For example, where a clause makes one party liable without limit, the suggestion caps the liability. The user can take the suggestion to the other party as a proposed amendment.

Technical description: The rewrite-clause endpoint receives the clause text and returns a safer version. In the primary path the language model is prompted to rewrite the clause so as to remove the identified risk while preserving its commercial purpose.

Semantic Search and Question Answering

User perspective: The user types a question about a contract in ordinary language, for example how the agreement can be terminated. The platform answers in a few sentences based on the contract and shows the passages from which the answer was drawn. The user can also search for passages similar to a phrase.

Technical description: The embed endpoint chunks the contract into pieces of 1000 characters with 200 characters of overlap, embeds each chunk into a 1536-dimensional vector and upserts the vectors into contract_embeddings, where the pair of contract and chunk index is unique. The search endpoint embeds the query and retrieves the top-K chunks, five by default, ordered by cosine distance using the HNSW index. Hybrid search adds a keyword score. The rag-query endpoint passes the retrieved chunks and the question through a LangChain prompt to GPT-4o-mini and returns a structured response, storing the exchange in rag_sessions. The VECTOR_SEARCH and RAG_QUERY activity types record these actions.

PDF Audit Reports

User perspective: From the report page the user generates a PDF for an analysed contract and downloads it. The report presents the contract, its score and level, the risky clauses, the findings and the suggested alternatives in a form suitable for sharing with a colleague, a client or a lawyer.

Technical description: The generate endpoint calls the report service, which builds the PDF with ReportLab from the stored analysis; the download endpoint returns the file. The REPORT_GENERATE activity type records the action. The Spring Boot report service offers enterprise report generation and report metadata storage as a separate microservice behind the gateway.

Analytics Dashboard

User perspective: The dashboard shows the user's totals, namely contracts uploaded, analyses run and average risk, together with the distribution of contracts by risk level, the riskiest contracts and how risk has changed over time.

Technical description: The analytics endpoints run SQL against PostgreSQL. The dashboard figures come from multi-table joins with aggregates, also available through the v_contract_dashboard view, which uses LEFT JOIN, GROUP BY and the FILTER clause. Rankings use window functions partitioned by user, as in the v_risk_summary view. Monthly statistics and percentages come from nested common table expressions in v_analytics_overview. The stored function fn_user_risk_stats returns a user's totals and counts by level in one call, and fn_top_risky_clauses returns the clause titles that are most often risky across the system.

Activity Tracking and Real-Time Events

User perspective: The history page shows what the user has done: logins, uploads, analyses, searches and reports. New activity appears in the interface as it happens.

Technical description: Activity is recorded in three ways. Database triggers write audit rows for uploads and completed analyses. The backend's activity service writes to PostgreSQL and, in a FastAPI background task, to the MongoDB activity_logs collection. The Node.js activity service receives events over REST, stores them through Mongoose and emits them over Socket.io to connected clients. The v_activity_audit view joins the logs to users for a full audit trail.

Secure Access

User perspective: The user registers with an email address and password and signs in. They remain signed in as they work, without being asked for their password repeatedly, and they can see only their own contracts.

Technical description: Passwords are hashed with pbkdf2_sha256. Login returns a 30-minute JWT access token and a 14-day opaque refresh token whose SHA-256 hash is stored. The frontend exchanges the refresh token for a new pair when the access token expires; the old refresh token is revoked. The gateway validates the JWT and

applies its policies, and the backend applies `require_role`. Token bucket rate limiting restricts each client to 100 requests per 60 seconds.

Resilience and Observability

User perspective: The user experiences these features as reliability. An analysis of a document that has been analysed before returns at once. Analysis still works when the AI provider cannot be reached. A fault in one part of the system does not make the whole application unresponsive.

Technical description: The analysis cache avoids repeated calls to the language model for identical documents. The heuristic engine and TF-IDF embeddings replace the external models when no key is present. The circuit breaker in the gateway opens after repeated failures of an upstream service and closes again after a successful probe. Background tasks keep logging off the request path. The backend exposes a requests counter and a latency histogram at `/metrics` for Prometheus, adds a request identifier and response time to every response, and can emit OpenTelemetry traces.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

API testing verifies that each endpoint accepts valid requests, rejects invalid ones and returns responses of the documented shape. The report template names Postman for this purpose. In this project the role of an API client was filled in two ways. For interactive, manual testing the team used the Swagger interface that FastAPI generates from the OpenAPI description of the backend, which allows any endpoint to be called from the browser with a token attached; a Postman collection can be produced from the same OpenAPI document by importing it. For repeatable verification the endpoints are exercised by the automated pytest suite. The report does not claim a separately maintained Postman collection.

Manual testing follows the order in which a user meets the API. The tester calls the signup endpoint and confirms that a token pair is returned, then the login endpoint, then the me endpoint with the access token to confirm that the token identifies the user. The refresh endpoint is called to confirm that a new pair is returned and that the old refresh token no longer works.

The contract endpoints are tested next: a file of each supported type is uploaded, the list endpoint is called to confirm the contract appears, the detail endpoint is called for it and the same file is uploaded again to observe the deduplication. The analyze endpoint is then called and its response examined for the score, the level, the clauses and the flags. The vector endpoints are tested by embedding the contract, searching it and submitting a question. The report endpoints are tested by generating and downloading a PDF.

Negative cases are tested alongside the positive ones. A request without a token must be refused. A request with a malformed body must return status 422 with the validation details. A signup with an email address that already exists must return the EMAIL_ALREADY_REGISTERED code. In every failing case the response is checked for the unified error envelope with its code, message, details and request identifier.

The response headers are inspected as well. Every response should carry the X-Request-ID and X-Response-Time-Ms headers added by the metrics middleware, and responses that pass through the gateway should carry the X-Gateway header. The health endpoint is called to confirm that it reports the state of the database connection and the MongoDB ping, and the metrics endpoint is called to confirm that the request counter and latency histogram are exposed. The areas covered by manual API testing are summarised in Table 8.1.

Area	Requests exercised	What is checked
Authentication	signup, login, refresh, me	Token pair issued; rotation; profile returned
Contracts	upload, list, detail, delete	Parsing of each file type; deduplication; ownership
Analysis	analyze, list, detail, rewrite-clause	Score, level, clauses, flags; safer wording
Vector and RAG	embed, search, rag-query, chunks	Chunks stored; relevant results; grounded answer
Analytics	dashboard and SQL endpoints	Figures consistent with stored data
Reports	generate, download	PDF produced and retrievable
Errors	Invalid, unauthenticated and duplicate requests	Status codes and the error envelope
Operations	health, metrics	Connectivity status; Prometheus metrics

Table 8.1: Manual API testing through the Swagger interface

8.2 Integration Testing

Integration testing verifies that the parts of the system work together. In a platform with four services and two databases, most of the interesting behaviour lies at the boundaries.

The first group of integration tests concerns the backend and PostgreSQL. A test that uploads a contract through the API and then reads it back verifies the router, the service, the repository, the ORM mapping and the schema together. The same test path verifies the triggers: after an upload, the activity log must contain the row that the trigger inserted, and after an analysis it must contain the completion row with its risk metadata.

The second group concerns the analysis pipeline. The test suite covers the AI analysis service and the clause detection service. Because the heuristic path is deterministic and needs no external service, it allows the pipeline to be tested end to end in an automated environment: a contract text containing known risky wording must produce the expected flags, a score computed by the stated rule and the level given by the thresholds.

The third group concerns the vector store. Tests verify that text is chunked with the configured size and overlap, that embeddings are stored and that a search returns the chunks nearest to the query.

The fourth group concerns authentication across requests: a token obtained at login must be accepted by a protected endpoint, and the refresh flow must rotate the token.

The fifth group concerns the analytics endpoints, whose SQL is run against data created in the test and whose results are compared with what that data implies.

The Node.js activity service has its own integration tests with Jest, which exercise the activity endpoints against a mocked MongoDB.

The integration of the whole system is verified by the CO1-CO6 verification script. This is a custom Python script that performs 28 checks with live assertions against the running system, covering all six course outcomes. It

confirms, for example, that the relational features exist and behave as expected, that the document and vector stores respond, that the API and authentication work, that the services are reachable, that the design patterns are in effect and that the operational endpoints respond.

Integration at the level of containers is verified by starting the Docker Compose stack. The health checks and the dependency ordering confirm that each service can reach the others on the shared network: the backend does not become healthy unless it can reach PostgreSQL and MongoDB.

Finally, the frontend's integration with the API is verified by the TypeScript compiler, which checks that the application uses the response types consistently, and by a clean production build.

8.3 Tools Used for Testing

pytest is the test framework of the core backend, used with the pytest-asyncio plugin so that asynchronous route handlers, services and database calls can be awaited inside tests. Fixtures prepare the application, the database session and test users.

Jest is the test framework of the Node.js activity service. It runs the service's tests with MongoDB mocked, so that the tests do not depend on a running database.

The custom verification script, written in Python, checks the six course outcomes against the live system.

The TypeScript compiler, run in strict mode, and the Vite build verify the frontend.

The Swagger interface generated by FastAPI is used for manual API testing. Docker Compose provides the environment in which the complete system is tested, and its health checks act as a continuous smoke test. Prometheus and Grafana allow the behaviour of the backend under test to be observed through its request counts and latencies. GitHub Actions runs the lint and test stages automatically in the pipeline. The tools are summarised in Table 8.2.

Tool	Used for
pytest, pytest-asyncio	Backend unit and integration tests
Jest	Tests of the Node.js activity service
Custom Python verification script	Live checks of CO1 to CO6
TypeScript compiler, Vite	Strict type check and build of the frontend
Swagger UI (OpenAPI)	Manual API testing
Docker Compose health checks	Container-level integration
Prometheus, Grafana	Observation of request counts and latency
GitHub Actions	Automated lint and test in the pipeline

Table 8.2: Tools used for testing

8.4 Test Cases and Results

The results of testing are summarised in Table 8.3. The backend suite contains 34 tests, the Node.js suite contains 4 tests, the verification script performs 28 checks and the frontend builds with no type errors.

Suite	Tool	Count	Coverage areas
Backend unit and integration	pytest + pytest-asyncio	34 tests	Auth, contracts, vector store, analytics, AI analysis, clause detection
Node.js activity service	Jest	4 tests	Activity endpoints with MongoDB mock
CO1-CO6 verification	Custom Python script	28 checks	All six course outcomes with live assertions
Frontend build	TypeScript + Vite	0 errors	Strict type check and clean build

Table 8.3: Testing summary

The backend tests are grouped by the area they cover. Representative test cases for each area are given in Table 8.4. They describe the behaviours that the suite verifies.

Area	Representative test case	Expected result
Auth	Sign up with a new email address	User created; access and refresh tokens returned
Auth	Sign up with an existing email address	Error with code EMAIL_ALREADY_REGISTERED
Auth	Log in with valid and invalid credentials	Tokens returned; invalid credentials refused
Auth	Refresh with a valid refresh token	New token pair; old refresh token revoked
Auth	Call a protected endpoint without a token	Request refused
Contracts	Upload a supported file	Contract stored with extracted text
Contracts	Upload the same content again	Duplicate detected through SHA-256
Contracts	List and fetch contracts	Only the caller's contracts returned
Clause detection	Split a multi-clause text	Clauses returned in order
AI analysis	Analyse text containing risky wording (heuristic)	Expected risk flags raised
AI analysis	Score and classify	Score = 15 + weights, capped at 100; level by thresholds
Vector store	Chunk a long text	Chunks of the configured size and overlap
Vector store	Search for a query	Nearest chunks returned in order of similarity
Analytics	Call dashboard and SQL endpoints	Results consistent with the stored data

Table 8.4: Representative backend test cases

The checks of the verification script are organised by course outcome, as shown in Table 8.5.

Outcome	What the live checks confirm
CO1	Tables, constraints, indexes, views, triggers, functions and advanced queries are present and return results
CO2	MongoDB collections respond; embeddings are stored; vector search and the RAG pipeline run
CO3	API endpoints, validation, authentication and the error envelope behave as specified
CO4	The services are present with their own responsibilities and respond
CO5	Caching, rate limiting, the circuit breaker and background execution are in place
CO6	Container, manifest and pipeline definitions exist; metrics endpoint responds

Table 8.5: CO1-CO6 verification by course outcome

The boundary values of the scoring rule deserve particular mention, because they determine the level shown to the user. They are given in Table 8.6.

Score	Expected level
0 to 34	LOW
35 to 64	MEDIUM
65 to 84	HIGH
85 to 100	CRITICAL

Table 8.6: Risk level thresholds

Some limits of the testing should be recorded. The language-model path depends on an external service whose output is not deterministic, so automated tests rely on the heuristic path for exact assertions. The system has not been load tested, so no claim is made about throughput. No formal security penetration test has been performed. And the quality of the risk assessment as legal judgement has not been evaluated against a set of contracts reviewed by lawyers. Within these limits, the tests show that the platform's components behave as designed and work together.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The platform is deployed as a set of containers. The recommended way to run the whole system is Docker Compose, which starts the databases, the services, the frontend and the monitoring tools with one command. The services can also be run individually for development. Both procedures are described here.

Deployment with Docker Compose proceeds as follows.

The first step is to install the prerequisites: Docker with Docker Compose, and Git.

The second step is to clone the repository from GitHub.

The third step is to create the environment configuration. The variables listed in Section 9.2 are set, in particular the JWT secret and, if the language-model path is wanted, the OpenAI API key. Without the key the platform runs with its heuristic analysis and TF-IDF embeddings.

The fourth step is to build and start the stack with Docker Compose. Docker builds an image for each service from its Dockerfile and starts the containers on the contract-guardian-net network.

The fifth step happens automatically. The PostgreSQL container starts from the pgvector/pgvector:pg16 image and, on its first start, runs the SQL scripts mounted into its initialisation directory in order: the schema with its tables, enumerated types and extensions; the indexes; the views, triggers and functions; and the advanced queries. The MongoDB container starts with its data volume. The other containers wait for the databases to report healthy.

The sixth step is also automatic. The backend container runs its start command, which applies the Alembic migrations and then starts Gunicorn with two Uvicorn workers on port 8000. At start-up the backend creates the MongoDB indexes for its three collections.

The seventh step is verification. The health endpoint of the backend is requested to confirm database connectivity and the MongoDB ping. The frontend is opened in a browser on port 8080. The Swagger documentation of the backend is opened to confirm the API is available. Prometheus is opened on port 9090 to confirm that it is scraping the backend, and Grafana on port 3000 to view the dashboards.

The eighth step is a functional check: a user is registered, a contract is uploaded and analysed, and a report is generated. The steps are summarised in Table 9.1.

Step	Action	Verification
1	Install Docker, Docker Compose and Git	Commands available
2	Clone the repository	Source present
3	Set environment variables	JWT secret set; OpenAI key optional
4	Build and start the Compose stack	Containers created on the bridge network
5	Databases initialise	PostgreSQL and MongoDB report healthy; SQL scripts executed
6	Backend migrates and starts	Gunicorn running; health check passes
7	Open the frontend, API docs, Prometheus and Grafana	Each responds on its port
8	Register, upload, analyse and generate a report	End-to-end flow succeeds

Table 9.1: Deployment with Docker Compose

For development, the services can be run directly. PostgreSQL and MongoDB are started first, most simply as containers. The backend is run from the backend directory in a Python 3.11 virtual environment after its dependencies are installed and the migrations applied; it serves on port 8000. The activity service is run from its directory with Node.js 20 after its packages are installed; it serves on port 3001. The report service is built and run with Maven on port 8082. The gateway is run with the .NET 8 SDK on port 5000, configured with the addresses of the three upstream services. The frontend is run from the frontend directory with the Vite development server on port 5173.

For a cluster, the Kubernetes manifests are applied. The ConfigMaps and Secrets are created first, then the Deployments and Services, and finally the Ingress. The probes defined in the Deployments let the cluster determine when each pod is ready.

In the automated path, the GitHub Actions pipeline performs the build and release: it lints, runs the tests, builds the images, pushes them to the registry and deploys.

9.2 Environment Configuration

All configuration is supplied through environment variables. The backend reads them into a Pydantic Settings object at start-up, with more than thirty parameters in total. The same image can therefore be run in different environments by changing its environment. The principal variables are listed in Table 9.2.

Variable	Example value	Purpose
DATABASE_URL	postgresql+asyncpg://user:pass@localhost:5432/contract_guardian	PostgreSQL connection for the async engine
MONGODB_URL	mongodb://localhost:27017	MongoDB connection
JWT_SECRET	(secret)	Signing key for access tokens

Variable	Example value	Purpose
ACCESS_TOKEN_EXPIRE_MINUTES	30	Lifetime of the access token
REFRESH_TOKEN_EXPIRE_DAYS	14	Lifetime of the refresh token
OPENAI_API_KEY	(secret, optional)	Enables the LLM and OpenAI embeddings
OPENAI_MODEL	gpt-4o-mini	Model used for analysis and RAG
EMBEDDING_MODEL	text-embedding-3-small	Model used for embeddings
EMBEDDING_DIMENSIONS	1536	Dimension of the vector column
CHUNK_SIZE	1000	Characters per chunk
CHUNK_OVERLAP	200	Characters of overlap between chunks
VECTOR_SEARCH_TOP_K	5	Default number of search results
OTEL_ENABLED	false	Enables OpenTelemetry tracing
PROMETHEUS_ENABLED	true	Enables the metrics endpoint
RATE_LIMIT_REQUESTS	100	Requests allowed per window
RATE_LIMIT_WINDOW_SECONDS	60	Length of the rate-limit window

Table 9.2: Environment variables of the core backend

The variables fall into four groups. The connection variables tell the backend where its databases are. Inside the Compose network these use the container names as hosts; on a developer's machine they use localhost.

The security variables are the JWT secret and the token lifetimes. The secret must be a long random value and must be the same for every component that validates tokens: the backend that issues them, the gateway and the activity service. It must never be committed to the repository.

The AI variables select the models and control chunking and retrieval. Because the embedding dimension is fixed in the column type of the `contract_embeddings` table, the `EMBEDDING_DIMENSIONS` value must agree with the model in use. Changing the embedding model to one with a different dimension requires the column and its index to be rebuilt.

The operational variables switch tracing and metrics on or off and set the rate limit.

The other services have their own configuration. The gateway is configured with the addresses of its upstream services, namely FastAPI on port 8000, the Node.js activity service on port 3001 and Spring Boot on port 8082, with the database connection for Entity Framework Core and with the token validation parameters. The activity service is configured with its MongoDB connection and the JWT secret. The frontend is configured at build time with the address of the API.

In Kubernetes, non-sensitive values are placed in ConfigMaps and sensitive ones in Secrets, and both are injected into the pods as environment variables. The application code is the same in every case; only the source of the values differs.

Cross-origin resource sharing is configured in the backend through its list of permitted origins. In development this includes the Vite server's address; in a deployment it should be restricted to the address from which the frontend is served.

9.3 Hosting of Frontend and Backend

The frontend is hosted as static files. The Vite build produces HTML, JavaScript and CSS, and in the Compose stack these are served by Nginx in the cg-frontend container on port 8080. Since the application handles its routes in the browser, the web server is configured to return the application's entry page for any application path. The same build could be hosted on any static hosting service.

The backend is hosted as a container running Gunicorn, a production process manager, with the Uvicorn worker class, which runs the asynchronous FastAPI application. Two workers are started, and the timeout is 180 seconds. Each worker runs its own event loop, so the service can use more than one processor core while each worker handles many concurrent requests asynchronously.

The other services are hosted in the same way, each in its own container: the Node.js activity service, the Spring Boot report service and the .NET gateway. In a full deployment the gateway is the only service exposed to clients, and the others are reachable only on the internal network.

The databases are hosted as containers with named volumes. The postgres_data and mongodb_data volumes hold the data outside the containers, so that the data survives when a container is replaced. In a cloud deployment these could be replaced by managed database services, with the connection variables pointing to them.

Monitoring is hosted alongside the application. Prometheus scrapes the backend's metrics endpoint according to its configuration file, and Grafana reads from Prometheus and is provisioned through a mounted volume.

In Kubernetes, each service becomes a Deployment with a Service in front of it. Internal services use the ClusterIP type and externally reachable ones the LoadBalancer type, and the Ingress routes external traffic by path or host. Liveness and readiness probes are defined on the Deployments.

For any deployment exposed to the internet, HTTPS should be terminated at the ingress or load balancer so that tokens and documents are encrypted in transit, the secrets should be generated for that environment, and the list of permitted origins should be restricted.

The deployment verified for this report is the local Docker Compose stack. The Kubernetes manifests and the pipeline's deploy stage define the path to a hosted deployment; the report does not claim that the platform is currently running on a public production host.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Building a platform that combines a language model, three kinds of data store and four services in four languages presented challenges at every level. The principal ones are described here.

The first challenge was the dependence on an external language model. A model accessed over the network is slow compared with a database query, it charges for every call, it limits the rate of requests and it may be unreachable. It also requires an API key that is not always available, for example in an automated test environment or on an evaluator's machine. A platform whose central feature simply failed without the key would be fragile and impossible to demonstrate reliably.

The second challenge was the unpredictability of model output. A language model produces text, and the platform needs data: a list of clauses, each with a risk level, a score and a set of flags from a fixed vocabulary. A model asked informally for such a result may return prose, may return JSON with missing or renamed fields, or may wrap the JSON in commentary. Storing such output without checking would corrupt the analysis tables or cause failures later.

The third challenge was the variety of document formats. A PDF, a DOCX file and a text file hold their content in entirely different ways, and each requires its own method of extraction. The text that comes out also differs in quality, with irregular line breaks and spacing, and the clause boundaries must still be found in it.

The fourth challenge was the length of contracts. A contract can be much longer than the amount of text that can usefully be embedded as one vector or placed in one prompt. The text has to be divided, and dividing it carelessly cuts sentences and ideas in half, so that neither piece carries the meaning.

The fifth challenge was vector search performance. Comparing a query vector with every stored vector of 1536 dimensions is exact but grows linearly with the number of chunks. As contracts accumulate, an exhaustive comparison becomes too slow for an interactive search.

The sixth challenge was keeping two databases consistent. Activity is recorded in both PostgreSQL and MongoDB, and contracts in PostgreSQL are referred to from MongoDB documents. There is no transaction that spans both systems, so a write may succeed in one and fail in the other.

The seventh challenge was asynchronous programming. The backend uses asynchronous input and output throughout: an asynchronous SQLAlchemy engine, the asynchronous Motor driver and asynchronous HTTP calls. In this model a single blocking call stalls every request served by the same event loop, and database sessions must be scoped carefully to the request that uses them. Testing asynchronous code needs its own tooling.

The eighth challenge was the polyglot nature of the system. Four languages mean four toolchains, four dependency managers, four build processes and four sets of conventions. A JSON Web Token issued by the Python service

must be validated identically by the .NET gateway and the Node.js service, which requires the algorithm, the secret and the claims to agree exactly. Field naming differs between languages, with Python favouring snake_case and JavaScript camelCase.

The ninth challenge was start-up ordering. In a multi-container system the services start at different speeds. A backend that starts before PostgreSQL is ready to accept connections fails at start-up, and the pgvector extension must exist before any table with a vector column is created.

The tenth challenge was failure propagation. When an upstream service is down or slow, every request routed to it waits for a timeout. Under load, these waiting requests consume the gateway's resources, and a fault in one service degrades all of them.

The eleventh challenge was tracing a problem across services. When a request passes through the gateway to a service and on to a database, an error reported to the user must be matched to the right entries in several separate logs.

10.2 Solutions Applied

Each challenge was met with a specific design decision.

The dependence on the language model was addressed in three ways. A heuristic analysis engine built from regular-expression patterns with severity weights provides a complete analysis when no OpenAI key is configured, and a TF-IDF method provides embeddings. Results are cached in the MongoDB analysis_cache collection keyed by the SHA-256 hash of the document, so that an identical document is never analysed twice. And the model is configured by environment variable, so it can be changed without touching the code. The heuristic engine has the further benefit of being deterministic, which made the analysis pipeline testable.

The unpredictability of model output was addressed by constraining and validating it. The prompt is built from a ChatPromptTemplate that states the JSON schema required. The request asks for a JSON object as the response format. The response is parsed with Pydantic's model_validate, which rejects anything that does not conform. Beyond that, the database itself refuses invalid values: the risk score must lie between 0 and 100 by a CHECK constraint, and the risk level must be a member of the enumerated type.

The variety of formats was addressed by a parser that routes each file by type to PyMuPDF, python-docx or UTF-8 decoding and returns plain text in every case. Everything after the parser works on text and is independent of the original format. Clause detection is isolated in its own service, so its patterns can be improved without affecting the rest.

The length of contracts was addressed by sentence-boundary chunking with overlap. Chunks of 1000 characters are cut at the ends of sentences, and consecutive chunks share 200 characters. Chunk-level SHA-256 hashes prevent identical chunks from being embedded twice, and the unique constraint on contract and chunk index allows a contract to be re-embedded by upsert.

Vector search performance was addressed with an HNSW index on the embedding column using cosine operators, with $m=16$ and $ef_construction=64$. HNSW is an approximate nearest-neighbour method that finds the closest vectors by navigating a layered graph instead of scanning every row. Hybrid search adds a keyword component for queries that need literal matches.

Consistency between the databases was addressed by assigning clear roles. PostgreSQL is the system of record; the primary write goes there, inside a transaction. The write to MongoDB is secondary and is performed in a background task, so that its failure does not fail the user's request. The most important audit entries, for uploads and completed analyses, are written by database triggers in the same transaction as the event they record and cannot be missed. MongoDB documents hold only references to PostgreSQL records.

The demands of asynchronous programming were met by using asynchronous libraries at every point of input and output, namely the `asyncpg` driver through SQLAlchemy's `AsyncSession`, `Motor` for MongoDB, by providing sessions through dependency injection scoped to the request, and by moving non-essential work into FastAPI background tasks. `pytest-asyncio` allows the tests to await the same code.

The polyglot difficulties were met by standards and containers. The services agree on JWT with HS256 and a shared secret, on REST with JSON, and on a request identifier header. The Pydantic models declare camelCase aliases, so the API speaks JavaScript's convention while the Python code keeps its own. Each service has its own Dockerfile, which hides its toolchain behind a uniform interface: every service is built and run the same way, whatever its language.

Start-up ordering was addressed with health checks and dependency conditions in Docker Compose, so that each container waits for those it depends on to be healthy, and by mounting the SQL scripts into the PostgreSQL initialisation directory so that the extensions and schema are created in order on first start. The backend applies its migrations before starting to serve.

Failure propagation was addressed with the circuit breaker in the gateway. After failures of an upstream service pass the threshold, the circuit opens and requests fail fast; a probe request in the half-open state tests for recovery. Rate limiting in the backend and the activity service bounds the load any one client can impose.

Tracing across services was addressed by the request identifier. The middleware assigns an identifier to each request, returns it in the X-Request-ID header and includes it in the error envelope, and the gateway propagates it to upstream services. Structured logging, Prometheus metrics and optional OpenTelemetry tracing complete the picture. The issues and solutions are summarised in Table 10.1.

Issue	Solution applied
External LLM cost, latency and availability	Heuristic fallback; TF-IDF embeddings; SHA-256 analysis cache
Unstructured model output	JSON schema in the prompt; JSON response format; Pydantic validation; database constraints
Different document formats	Parser routing to PyMuPDF, python-docx or UTF-8 decoding
Long documents	Sentence-boundary chunks of 1000 characters with 200 overlap
Slow vector comparison	HNSW index with cosine operators; hybrid search
Two databases without a shared transaction	PostgreSQL as system of record; background write to MongoDB; triggers for audit rows
Blocking in asynchronous code	Async drivers throughout; request-scoped sessions; background tasks
Four languages and toolchains	Shared standards (JWT, REST, request ID); camelCase aliases; one Dockerfile per service
Start-up race conditions	Health checks and dependency ordering; ordered SQL initialisation
Cascading failures	Circuit breaker in the gateway; rate limiting
Tracing across services	Request ID propagation; structured logs; metrics; tracing

Table 10.1: Issues faced and solutions applied

10.3 Current Limitations or Known Bugs

The platform has limitations that a user or evaluator should know.

The analysis is an aid and not legal advice. The risk flags and scores indicate where attention is needed; they do not establish whether a clause is enforceable or acceptable in a particular jurisdiction or business context. The accuracy of the assessment has not been measured against a body of contracts reviewed by legal professionals.

The heuristic engine is limited by its patterns. It recognises risk by the wording it has been given patterns for. A risky clause phrased in an unusual way may be missed, and a harmless clause that happens to contain a matching phrase may be flagged. Its score is additive, so it reflects how many kinds of risk were found and not how severe each instance is.

The language-model path depends on an external provider, with its costs and limits. The same contract analysed twice by the model may produce slightly different results; the cache makes results stable for an identical document but not across different versions of it.

Documents that consist of scanned images are not supported. The parsers extract embedded text, and a scanned PDF without a text layer yields none. Optical character recognition is not implemented. The analysis patterns and prompts are written for English.

The TF-IDF fallback for embeddings captures shared vocabulary and not meaning. Semantic search is therefore weaker without the OpenAI embedding model. The embedding dimension is fixed in the schema at 1536, so a model with a different dimension cannot be adopted without rebuilding the column and index.

The access token is stored in the browser's localStorage, where it could be read by a script injected through a cross-site scripting flaw. Storing tokens in cookies that scripts cannot read would be more robust.

The rate limiter of the backend keeps its counters in the memory of each process. With two Gunicorn workers, and more so with several replicas, each process counts separately, so the effective limit across the service is higher than the configured one. A shared store would be needed for an exact global limit.

The write to MongoDB is best-effort. If the background write fails, the activity exists in PostgreSQL but not in MongoDB, and the two can differ until reconciled.

Several components are partial by design. The Pinecone and Weaviate adapters are stubs. Kubernetes support consists of manifests, without a claimed running cluster. The payments table is present in the schema without a payment provider integration. The documented Docker Compose stack lists the core backend, the activity service, the frontend, the two databases and the monitoring tools; the Spring Boot report service and the .NET gateway have their own Dockerfiles and are run alongside it.

The system has not been load tested or penetration tested, and it has no automated backup procedure for its volumes.

No specific defect in the implemented behaviour is recorded at the time of writing. The items above describe the boundaries of what the platform does.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The architecture of Contract Guardian AI was chosen so that it could grow. The enhancements described in this chapter follow from the limitations of the present system and from the direction in which contract analysis is developing.

The first planned feature is optical character recognition. Many contracts exist only as scans. Adding an OCR stage to the document parser, applied when a PDF yields no text layer, would bring these documents within the platform's reach. The rest of the pipeline works on text and would not need to change.

The second planned feature is support for more languages. Contracts are written in every language, and in many countries in more than one. The language-model path can analyse text in other languages with adapted prompts, and the heuristic patterns could be extended language by language.

The third planned feature is contract comparison. The schema already records versions of a contract in the `contract_versions` table. A comparison feature would show what changed between two versions and how the changes affect the risk, so that a user negotiating an agreement could see whether a revised draft is better or worse than the last.

The fourth planned feature is a clause library and playbooks. An organisation usually has standard positions: clauses it always accepts, clauses it never accepts and preferred wording. A library of approved clauses, with rules describing the organisation's positions, would allow the platform to compare an incoming contract with those positions and to suggest the organisation's own wording as the safer alternative.

The fifth planned feature is collaborative review. Analysts could comment on clauses, assign them to colleagues and mark them as accepted or disputed. The Socket.io channel that already carries live activity would carry these updates, so that several people could review one contract together.

The sixth planned feature is obligation tracking. Contracts contain dates and duties: renewal deadlines, notice periods, payment schedules. Extracting these and reminding the user before they fall due would extend the platform from review at signing to management across the life of the contract. The automatic renewal flag, already detected, is a natural starting point.

The seventh planned feature is a feedback loop. When an analyst corrects the platform, by marking a flagged clause as acceptable or flagging one that was missed, that judgement is valuable. Recording it would allow the heuristic weights and the prompts to be improved over time and the accuracy of the analysis to be measured.

The eighth planned feature is electronic signature and payment. Once a contract has been reviewed and agreed, signing it is the next step; integration with an e-signature provider would complete the workflow. The payments table in the schema anticipates subscription plans, which would require integration with a payment provider.

The ninth planned feature is stronger account security: storing tokens in cookies inaccessible to scripts, multi-factor authentication, password reset by email and single sign-on for enterprise customers.

The tenth planned feature is a mobile-friendly experience, allowing a user to photograph or upload a contract from a phone and read the summary of its risks before signing.

11.2 Possible Integrations or Optimizations

Alongside new features, the platform can be improved in how it performs and operates.

An asynchronous analysis pipeline would be the most significant optimisation. At present an analysis is requested and awaited. Placing analysis jobs on a message queue, with worker processes consuming them, would allow many contracts to be analysed in parallel, would let the user continue working while a long analysis runs and would allow failed jobs to be retried. Progress could be reported to the browser over the existing Socket.io channel. The `contract_status` values already model the stages of such a pipeline.

A distributed cache would improve consistency across replicas. Redis could hold the rate-limit counters, so that the limit is enforced globally and not per process, and could cache frequent analytics results and sessions.

The vector store can be scaled in two ways. The Pinecone and Weaviate adapter stubs can be completed, allowing the embeddings to be moved to a managed vector database if their number outgrows what is comfortable in PostgreSQL. Within pgvector, the HNSW parameters and the search-time effort can be tuned against measured recall and latency, and the table can be partitioned by user.

Retrieval quality can be improved by re-ranking: retrieving a larger set of candidate chunks by vector similarity and then ordering them with a more precise model before they are given to the language model. Chunking could also follow the structure of the contract, using the detected clauses as chunks, so that each retrieved passage is a complete clause.

The model layer can be made more flexible. Because the model is selected by configuration and accessed through LangChain, other providers or locally hosted open models could be supported. A locally hosted model would allow confidential contracts to be analysed without sending their text to an external service, which matters to many organisations. A model fine-tuned on legal text could improve accuracy.

Communication between services can be optimised. Internal calls could use gRPC in place of REST for lower overhead and strictly typed contracts. Events such as the completion of an analysis could be published to a message broker, so that the activity service and the report service react to them without being called directly.

Database operations can be strengthened. Read replicas of PostgreSQL could serve the analytics queries. Large tables such as `activity_logs` could be partitioned by date. Analytics views that are expensive to compute could become materialised views refreshed on a schedule. Scheduled execution of the token cleanup procedure and automated backups with tested restoration would complete the operational picture.

Deployment can be matured. The Kubernetes manifests can be packaged as a Helm chart and deployed to a managed cluster, with horizontal pod autoscaling driven by the Prometheus metrics the backend already exposes. A service mesh could provide encrypted communication and retries between services. The pipeline could add security scanning of images and dependencies and staged releases.

Observability can be extended. With OpenTelemetry enabled in every service, and not only in the backend, a trace would follow a request from the gateway through each service to the database. Centralised log aggregation would make the request identifier searchable across all services in one place. Alerts could be defined on error rates and latency.

Finally, the platform can integrate with the tools in which contracts live: cloud document storage, email, contract lifecycle management systems and messaging platforms, so that a contract can be sent for analysis from where it arrives and the result delivered where the team works.

In summary, the future development of the platform has three directions: broader coverage of documents and languages, deeper support for the work of reviewing and managing contracts, and greater maturity in scale and operation. The present architecture, with its separated services, its clear data ownership and its instrumented backend, provides the foundation for each.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

Contract Guardian AI is an enterprise-grade platform for the analysis of legal contracts. A user uploads an agreement as a PDF, DOCX or TXT file, and the platform extracts its text, divides it into clauses and chunks, detects high-risk clauses and deceptive patterns, computes a risk score from 0 to 100 with a level from LOW to CRITICAL, suggests safer wording, answers questions about the document and produces a PDF audit report.

The platform is built as a polyglot microservices system. A React single-page application written in TypeScript provides the interface. A .NET Core 8 gateway validates tokens and routes requests. A Python FastAPI service performs authentication, contract management, analysis, analytics and vector search. A Node.js service records activity and streams it in real time. A Spring Boot service generates enterprise reports. PostgreSQL with pgvector holds the relational data and the embeddings, and MongoDB holds activity logs, the analysis cache and retrieval sessions.

The analysis engine has two paths. The primary path uses GPT-4o-mini through LangChain with enforced JSON output validated by Pydantic. The fallback path uses regular-expression patterns with severity weights and needs no external service. Both produce the same structure, covering 26 risk flag categories including a group dedicated to fraud indicators.

The relational database is used to its full extent: a normalised schema with constraints and enumerated types, B-Tree, GIN and HNSW indexes, six views, four triggers, stored functions and a procedure, and a set of analytics endpoints that exercise joins, subqueries, common table expressions, window functions, aggregates and full-text search.

Security rests on hashed passwords, short-lived JWT access tokens, rotating refresh tokens stored only as hashes, three roles enforced in both the gateway and the backend, and rate limiting. Resilience rests on caching, the heuristic fallback, the circuit breaker and background tasks. Operation rests on Docker images for every service, a Docker Compose stack with health checks, Kubernetes manifests, a GitHub Actions pipeline, Prometheus metrics, Grafana dashboards and optional OpenTelemetry tracing.

The project was designed so that each of the six course outcomes is realised by a working part of one system. The relational database outcome is met by the PostgreSQL design; the NoSQL and vector outcome by MongoDB and pgvector; the backend outcome by the FastAPI application; the microservices outcome by the four services; the system design outcome by the caching, rate limiting, circuit breaker and asynchronous execution; and the DevOps outcome by the containers, manifests, pipeline and monitoring.

12.2 What was Achieved

The project achieved its primary objective. The platform takes a contract and reports, in a consistent and understandable form, where its risks lie and how they might be reduced.

In relational database engineering, the project delivered a schema in third normal form whose tables protect their own integrity. Email addresses are unique and checked; file sizes are positive; risk scores lie between 0 and 100; token expiry follows creation; statuses, levels and roles are restricted to enumerated values. Deleting a contract removes its analyses and embeddings through cascading foreign keys. Triggers maintain timestamps and write audit records without application code. Views provide dashboards, rankings and audit trails. Functions return a user's risk statistics and perform ranked full-text search, and a procedure cleans up expired tokens transactionally.

In advanced SQL, the project demonstrated inner, left and multi-table joins, subqueries and correlated subqueries, common table expressions, the window functions ROW_NUMBER, RANK, DENSE_RANK and NTILE, grouping with HAVING, UNION, aggregate functions including PERCENTILE_CONT, an ACID transaction with rollback and full-text search with ranking, each exposed as a working analytics endpoint.

In NoSQL and vector databases, the project delivered three indexed MongoDB collections used through an asynchronous driver, a vector table with an HNSW index, a chunking and embedding pipeline with deduplication, similarity and hybrid search, and a complete retrieval-augmented generation pipeline. It showed polyglot persistence in practice, with each kind of data in the store that suits it and references linking them.

In backend engineering, the project delivered an asynchronous FastAPI application with a clear separation of routers, services, repositories and schemas, validated contracts with camelCase aliases, a full authentication system with refresh token rotation, role-based access, a unified error envelope with request identifiers, automatic OpenAPI documentation and a suite of 34 automated tests.

In microservices, the project delivered four services in four languages, each with its own responsibility, dependencies and Dockerfile, communicating over REST and WebSocket, with centralised authentication at a gateway.

In system design, the project delivered a cache that prevents repeated analysis of identical documents, token bucket rate limiting, a circuit breaker with its three states and non-blocking background execution.

In DevOps and observability, the project delivered a Compose stack with networks, volumes, health checks and ordered start-up, Kubernetes manifests with probes, ConfigMaps, Secrets and Ingress, a CI/CD pipeline, a metrics endpoint with a request counter and latency histogram, and tracing support.

In applied AI, the project delivered structured, validated analysis from a language model, a deterministic fallback, fraud pattern detection, clause rewriting and question answering grounded in the document.

In the frontend, the project delivered a ten-page application with protected routes, a consistent dark design system in which colour conveys risk, and a strict, error-free TypeScript build.

Verification supports these achievements: 34 backend tests, 4 tests of the Node.js service, 28 live checks across the six course outcomes and a clean frontend build.

Perhaps the most valuable achievement is the integration. Each technology in the platform could be demonstrated in isolation with less effort. Making them work together, so that an upload flows through parsing, hashing, storage, triggers, analysis, caching, embedding, logging and reporting across services and databases, is what gives the project its substance.

12.3 Skills Learned During Development

The project developed skills across the full breadth of modern software engineering.

In database design, the team learned to normalise a schema and to express business rules as constraints, so that the database refuses invalid data instead of trusting the application. The team learned when to use each kind of index: B-Tree for equality and range, GIN for full-text and trigram search and HNSW for vectors. Writing triggers, views, functions and procedures in PL/pgSQL showed how much logic can live close to the data and when it should.

In SQL, the team moved beyond simple queries. Window functions made rankings and percentages possible without self-joins. Common table expressions made complex queries readable. Understanding transactions and PostgreSQL's multi-version concurrency control clarified what the isolation levels mean in practice.

In NoSQL and vector databases, the team learned to model documents by how they are read, to choose between embedding and referencing, and to decide which data belongs in which store. Working with embeddings taught what a vector represents, how cosine similarity measures closeness of meaning, how approximate nearest-neighbour search trades a little accuracy for a great deal of speed, and how chunk size and overlap affect retrieval.

In backend development, the team learned asynchronous programming in Python: the event loop, awaiting input and output and the danger of blocking calls. The team learned to structure an application in layers with dependency injection, to validate data at the boundary with Pydantic and to design errors as part of the API. Implementing authentication taught the reasons behind short-lived access tokens, rotating refresh tokens and storing only hashes.

In working with language models, the team learned prompt design, how to obtain structured output and why it must be validated, and how retrieval grounds a model's answer in a source. Equally important was learning to design around the model's weaknesses, with caching for cost and a fallback for availability.

In distributed systems, the team learned what changes when one program becomes several: services must agree on protocols and token formats, each must own its data, failures must be contained and a request must be traceable across process boundaries. Implementing the circuit breaker made the idea of failing fast concrete. Working in Python, JavaScript, Java and C# within one system broadened the team's fluency and showed that architectural ideas carry across languages.

In frontend development, the team learned TypeScript's type system, server state management with React Query, route protection and the building of a design system from tokens in Tailwind CSS.

In DevOps, the team learned to write Dockerfiles, to compose a multi-container system with health checks and dependency ordering, to describe a deployment in Kubernetes manifests and to automate linting, testing, building and release in a pipeline. Instrumenting the backend with Prometheus metrics and reading them in Grafana showed how a running system is understood through measurement.

In testing, the team learned to test asynchronous code, to mock external dependencies, to make AI-backed features testable through a deterministic path and to verify a whole system with live checks.

Beyond the technical skills, the project taught planning and judgement: dividing a large system into parts that can be built and tested independently, deciding what to implement fully and what to provide as a stub, and describing accurately what the system does and does not do.

In conclusion, Contract Guardian AI addresses a real need, the understanding of contractual risk by people who are not lawyers, and does so with an architecture that brings together relational, document and vector databases, a validated AI pipeline, four cooperating services and a complete operational toolchain. It meets the six outcomes of the course within one coherent product and leaves a sound base for the enhancements described in the previous chapter.

CHAPTER -13 REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Database Documentation

PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Retrieved from <https://www.postgresql.org/docs/16/>

pgvector. *Open-source vector similarity search for Postgres*. Retrieved from <https://github.com/pgvector/pgvector>

MongoDB Inc. *MongoDB Manual*. Retrieved from <https://www.mongodb.com/docs/manual/>

Motor. *Asynchronous Python driver for MongoDB*. Retrieved from <https://motor.readthedocs.io>

Backend and Service Frameworks

FastAPI. *FastAPI Documentation*. Retrieved from <https://fastapi.tiangolo.com>

SQLAlchemy. *SQLAlchemy 2.0 Documentation*. Retrieved from <https://docs.sqlalchemy.org>

Pydantic. *Pydantic Documentation*. Retrieved from <https://docs.pydantic.dev>

Express. *Express Documentation*. Retrieved from <https://expressjs.com>

Socket.IO. *Socket.IO Documentation*. Retrieved from <https://socket.io/docs/>

Spring. *Spring Boot Reference Documentation*. Retrieved from <https://spring.io/projects/spring-boot>

Microsoft. *ASP.NET Core Documentation*. Retrieved from <https://learn.microsoft.com/aspnet/core>

AI and Document Processing

OpenAI. *OpenAI API Documentation*. Retrieved from <https://platform.openai.com/docs>

LangChain. *LangChain Documentation*. Retrieved from <https://python.langchain.com>

PyMuPDF. *PyMuPDF Documentation*. Retrieved from <https://pymupdf.readthedocs.io>

ReportLab. *ReportLab Documentation*. Retrieved from <https://docs.reportlab.com>

Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33.

Frontend

React. *React Documentation*. Retrieved from <https://react.dev>

Vite. *Vite Guide*. Retrieved from <https://vite.dev>

Tailwind CSS. *Tailwind CSS Documentation*. Retrieved from <https://tailwindcss.com/docs>

DevOps and Observability

Docker. *Docker Documentation*. Retrieved from <https://docs.docker.com>

Kubernetes. *Kubernetes Documentation*. Retrieved from <https://kubernetes.io/docs/>

GitHub. *GitHub Actions Documentation*. Retrieved from <https://docs.github.com/actions>

Prometheus. *Prometheus Documentation*. Retrieved from <https://prometheus.io/docs/>

OpenTelemetry. *OpenTelemetry Documentation*. Retrieved from <https://opentelemetry.io/docs/>

Standards

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*, RFC 7519. Internet Engineering Task Force. Retrieved from <https://www.rfc-editor.org/rfc/rfc7519>

CHAPTER -14 APPENDICES

Screenshots of the app

[Insert screenshots of the application here: landing page, dashboard, upload page, analysis result with risk score and flagged clauses, and the generated PDF report.]

Sample code snippets

The structure of the FastAPI application is shown below.

```
backend/app/
main.py          # App factory, middleware, startup/shutdown, Prometheus, error handlers
core/
  config.py      # Pydantic Settings (env-based), 30+ config parameters
  constants.py   # Enums, RISK FLAGS set (26 flags), risk level from score()
  security.py    # JWT encode/decode, pbkdf2_sha256 hashing, refresh tokens
  errors.py      # Typed AppError with HTTP status + error code
  logging.py     # Structured logging configuration
db/
  models.py      # SQLAlchemy 2.0 ORM models
  session.py     # Async SQLAlchemy engine + AsyncSession factory
  mongo.py       # Motor async client, 3 collection accessors, indexes
  base.py        # DeclarativeBase
api/v1/
  router.py      # Aggregates all sub-routers
  auth.py        # POST /signup, /login, /refresh, GET /me
  contracts.py   # CRUD for contracts, file upload, SHA-256 dedup
  analysis.py    # POST /analyze, GET /analyses, rewrite-clause
  analytics.py   # 15+ SQL analytics endpoints
  vector.py      # Embed, search, RAG query endpoints
  activity.py    # MongoDB activity log endpoints
  reports.py     # PDF report generation
  health.py      # Health check endpoint
services/
  ai_analysis.py # LLM + heuristic analysis, risk flag patterns
  clause_detection.py # Regex-based clause splitting
  vector_store.py # Chunking, embedding, HNSW search, RAG
  auth.py        # Signup, login, refresh token rotation
  contracts.py   # File parsing, SHA-256, storage
  document_parser.py # PDF/DOCX/TXT text extraction
  activity.py    # MongoDB + PostgreSQL dual-write
  analysis.py    # Orchestrates AI + DB persistence
  report.py      # ReportLab PDF generation
schemas/        # Pydantic v2 models with camelCase aliases
repositories/   # UserRepository, ContractRepository, etc.
```

All errors are returned in the following JSON envelope.

```
{
  "error": {
    "code": "EMAIL_ALREADY_REGISTERED",
    "message": "Email is already registered",
    "details": {},
    "requestId": "req abc123def456"
  }
}
```

The backend container starts with the following production command.

```
alembic upgrade head && gunicorn app.main:app \
-k uvicorn.workers.UvicornWorker -b 0.0.0.0:8000 --workers 2 --timeout 180
```

The vector index is created with the HNSW method and cosine operators.

```
contract_embeddings USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64)
```

[Insert screenshots of further source code here, for example ai_analysis.py, vector_store.py and 03_views_triggers_functions.sql.]

The key files of the project are listed in Table 14.1.

File	Path	Description
Main entry	backend/app/main.py	FastAPI app factory, middleware, startup, Prometheus, error handlers
ORM models	backend/app/db/models.py	SQLAlchemy 2.0 models with relationships
AI analysis	backend/app/services/ai_analysis.py	LLM and heuristic analysis engine
Vector store	backend/app/services/vector_store.py	Chunking, embeddings, HNSW search, RAG pipeline
Auth service	backend/app/services/auth.py	Signup, login, refresh token rotation
Security	backend/app/core/security.py	JWT, password hashing, SHA-256
Config	backend/app/core/config.py	Configurable settings
Constants	backend/app/core/constants.py	Enums, RISK_FLAGS, risk_level_from_score()
MongoDB	backend/app/db/mongo.py	Motor client, collections, index creation
SQL schema	database/sql/01_schema.sql	Tables, ENUMs, extensions, catalogue view
Indexes	database/sql/02_indexes.sql	B-Tree, GIN, HNSW indexes
Views and triggers	database/sql/03_views_triggers_functions.sql	Views, triggers, functions and procedures
Advanced SQL	database/sql/04_advanced_queries.sql	Query demonstrations
Analytics API	backend/app/api/v1/analytics.py	SQL analytics REST endpoints
Node service	services/node-activity/src/index.js	Express, Mongoose, Socket.io, Winston
.NET gateway	services/dotnet-gateway/Program.cs	ASP.NET Core, JWT, EF Core, routing
Docker Compose	docker-compose.yml	Container stack
App router	frontend/src/App.tsx	React Router setup, 10 routes
Tailwind config	frontend/tailwind.config.ts	Custom design tokens
Dependencies	backend/pyproject.toml	Python dependencies

Table 14.1: Key files

Installation/setup instructions

Contract Guardian AI is most easily installed with Docker. Install Docker with Docker Compose and Git, then clone the repository.

Create the environment configuration with the variables described in Chapter 9. At a minimum set `JWT_SECRET` to a long random value. Set `OPENAI_API_KEY` to use GPT-4o-mini and OpenAI embeddings; leave it unset to use the heuristic analysis and TF-IDF embeddings.

Build and start the stack with Docker Compose. On first start the PostgreSQL container creates the extensions, schema, indexes, views, triggers and functions from the SQL scripts, and the backend applies its migrations before it begins serving.

When the containers are healthy, open the frontend at port 8080, the backend's API documentation at port 8000, Prometheus at port 9090 and Grafana at port 3000. The backend's health endpoint reports the status of the PostgreSQL and MongoDB connections.

To run the services without containers for development, start PostgreSQL (with pgvector) and MongoDB, then start the backend with Python 3.11 on port 8000, the activity service with Node.js 20 on port 3001, the report service with Java 17 and Maven on port 8082, the gateway with the .NET 8 SDK on port 5000 and the frontend with the Vite development server on port 5173.

User manual or guide

Open the application in a browser. The landing page introduces the platform. Choose Register to create an account with your email address and a password, or Login if you already have one.

After signing in you arrive at the Dashboard, which shows your contracts, analyses and risk overview. To analyse a contract, open the Upload page, choose a PDF, DOCX or TXT file and submit it.

Open the Analysis page and run the analysis for the uploaded contract. The result shows the risk score from 0 to 100, the risk level, the clauses that were flagged, the reason for each flag and a suggested safer wording. Green indicates low risk, amber medium, and red high or critical.

To ask a question about the contract, enter it in ordinary language; the answer is drawn from the contract's own text. Open the Report page to generate and download the PDF audit report.

The History page lists your earlier contracts and activity. The Profile and Settings pages show and adjust your account details. Use the logout option when you have finished.

The platform's output is an aid to review. For decisions with legal consequences, consult a qualified legal professional.

Geo Tag photos with guide

[Insert geo-tagged photo of the team with the project guide here.]

Review forms with guide signatures

[Insert scanned review and evaluation forms signed by the guide here.]